MLFF Training-Data and Fine-Tuning Architecture

Purpose

This manual defines the current scientific, statistical, execution, and evidence architecture for the mdstats MLFF training-data package. It covers source-certified atomistic data preparation, leakage-safe partitioning, target-data construction, MACE fine-tuning, evaluation, deployment verification, and the campaign performance architecture.

The manual is intentionally **state-oriented rather than revision-oriented**. Historical release deltas, architecture revision notes, and patch notes are retained under `docs/history/mlff/`; they do not define current scientific contracts. The complete revision-90 predecessor is retained in `docs/history/mlff/manual_snapshots/`.

Architectural motive

MLFF campaigns mix several kinds of state that must not be conflated: physical source facts, eligibility decisions, statistical partitions, fitted transforms, subset-selection decisions, optimization/checkpoint state, evaluation evidence, and deployment decisions. The architecture therefore uses immutable, content-addressed records and explicit ownership boundaries. The same separation is applied to performance: execution caches, schedulers, worker counts, and memory layouts may change without silently changing scientific authority.

The current performance roadmap has one additional motive: **expensive exact numerical work should be computed once, exposed as enough independent tasks to occupy the allowed hardware, and reused downstream whenever its semantic inputs are unchanged**. This principle is applied first to the exact TARGET-DATA2B neighborhood graph shared by FEAS1 and MVIDX1.

Canonical documentation layout

The release-facing authority is this assembled file and its synchronized PDF:

- `docs/arch_manuals/mlff_training_data_architecture.md`
- `docs/arch_manuals/mlff_training_data_architecture.pdf`

The source chapters are maintained under `docs/arch_manuals/mlff_training_data/` and assembled deterministically by `tools/build_mlff_architecture_manual.py`. This split is for navigation and contextual loading only; chapter files must not contradict the assembled authority.

Historical lineage is non-normative and is stored under:

- `docs/history/mlff/architecture_revisions/`
- `docs/history/mlff/release_notes/`
- `docs/history/mlff/manual_snapshots/`

Reading index

Need	Primary chapter
Physical/statistical motivation and scope	Part I - Foundations and ownership
Source, labels, strain/stress, eligibility, feature/event contracts	Part II - Data and evidence contracts
Leakage control, cross-validation, selection, objective weighting	Part III - Statistical design and selection
Replay, MACE adapter, training/evaluation, active learning, determinism	Part IV - Training and evaluation
FEAS1/MVIDX1/MVSEL1/REPAIR1/MVQUAL1 theory and exact multi-view graph	Part V - Multi-view target-data architecture
Shared scheduler, vectorization, cache reuse, NUMA/memory policy, progress	Part VI - Performance and execution architecture
Current implementation state, frozen optimization gates, acceptance	Part VII - Status and forward gates
External scientific/algorithmic sources	References

Context retrieval index

For targeted human or AI loading, use the smallest authoritative source that contains the needed contract:

Query terms	Load first
`DATA*`, source/label identity, eligibility, stress/strain, features	`20_data_contracts.md`
partition, leakage, CV, selection, weighting, exposure	`30_statistical_design.md`
replay, MACE, checkpoint, evaluation, active learning, determinism	`40_training_evaluation.md`
FEAS1, MVIDX1, MVSEL1, REPAIR1, MVQUAL1, target rungs	`50_target_multiview.md`
scheduler, utilization, CSR/CSC, vectorization, NUMA, progress	`60_execution_performance.md`
`PERFBASE1` through `MVSTATE-REUSE1`, current status	`70_status_and_gates.md`
ownership or current design decision	`80_ownership_and_decisions.md`
provenance for an algorithmic/scientific idea	`90_references.md`
why/when a historical decision changed	MLFF revision index under `docs/history/`

The assembled manual is the release-facing authority; chapter files are retrieval units, not independent competing specifications.

Normative vocabulary

- **SHALL / MUST**: required for scientific or execution correctness.
- **SHOULD**: default design unless measured evidence justifies another exact-equivalent realization.
- **MAY**: optional realization that cannot weaken scientific contracts.
- **authoritative evidence**: persisted information that defines or proves a scientific decision.
- **reconstructible execution cache**: discardable state derivable exactly from authoritative inputs.

Current release boundary

`mdstats 0.20.241a0` is an exact-execution MVSEL production-density hardening release on architecture revision 103. Initialization gathers FP64 weights in complete-row chunks bounded to a 512 MiB target and releases scanned inverse-mmap pages. Sparse scatter validation retains one touched bit per candidate rather than every processed edge. Families with at least 98% rectangular density use canonical witness-ordered contiguous-run adds that are bit-exact to the scalar oracle. On the supplied 36,408-candidate, 165-family, 9.51-billion-edge domain, scoped initialization plus rank 0 improves from about 320 s to 106 s and peak RSS from about 51.5 GiB to 19.2 GiB. Rate-limited initialization/rank heartbeats are now emitted before the first rung. Scientific identity, sequential rank authority, revision 103, schema 83, and `FINAL-GPU1` remain unchanged.

`mdstats 0.20.240a0` is an exact-execution MVIDX/PARCORE1 backpressure hardening release on architecture revision 103. MVIDX no longer eager-submits every required family inversion into the bounded PARCORE1 ready queue. It feeds family and hard-obligation tasks through a deterministic producer/consumer refill loop: submit only while ready capacity exists, wait for completion, drain canonical completions, then refill. This preserves bounded ready/in-flight/completed queues and explicit RAM admission while allowing domains with arbitrarily more required families than queue slots (including the observed 165-family / 56-ready-slot production case). Scientific sparse-index authority, out-of-core storage semantics, worker-independent digests, and `FINAL-GPU1` as the next scientific gate are unchanged.

`mdstats 0.20.239a0` is a Python-3.11 compatibility hotfix on architecture revision 103. It corrects the DATA6 progress reporter so canonical timing fields are computed before f-string interpolation; the 0.20.238a0 MVIDX out-of-core execution/storage hardening and all scientific authority are unchanged. Release qualification now includes whole-tree Python 3.11 grammar parsing. `FINAL-GPU1` remains the next scientific gate.

`mdstats 0.20.238a0` is an exact-equivalence MVIDX scaling-hardening release on architecture revision 103. Multi-billion-edge NEIGHBOR1 caches may exceed the anonymous-RAM capacity required by the original full-family SciPy transpose even when the final scientific uint32/uint64 sparse arrays are valid. Campaign MVIDX therefore uses bounded row-chunk CSR-to-CSC transposes for large families, writes candidate-to-witness arrays directly as file-backed NPY memmaps, hard-links whole mmap-backed arrays into the authenticated native store on the same filesystem, and reloads the durable mmap authority before removing transient build paths. Queue admission accounts bounded transient scratch rather than the complete inverse payload for this path. A disk-space preflight fails before inversion if the exact inverse edge payload plus safety headroom cannot fit. In-memory and out-of-core arrays are required to be byte-identical and produce the same MVIDX content digest. This

hardening changes execution/storage realization only; revision 103, schema 83, scientific authority, MPA-0/MH-1 semantics, and the `FINAL-GPU1` next-gate decision remain unchanged.

`mdstats 0.20.237a0` is a presentation-only maintenance release on top of architecture revision 103. It standardizes MLFF progress and heartbeat output across preparation, TARGET-DATA2, model sweep, training, inference/evaluation schedulers, and qualification callbacks. Every elapsed/ETA field now uses fixed-width `HH:MM:SS`; unavailable ETA is `--:--:--`; progress counters, rates, phase/status fields, and semicolon delimiters follow the common Part VI observability contract. No scientific identity, scheduler authority, model-family behavior, dependency-graph node, or CPU/GPU gate decision changes. `FINAL-GPU1` remains next.

Revision 103 completes `MVSTATE-REUSE1` and closes the exact-equivalence CPU optimization program. MVSEL now emits authenticated exact sparse-state checkpoints at materializable target rungs; REPAIR consumes those checkpoints only while its state is still identical to MVSEL and falls back to the historical carried-forward arithmetic after the first accepted repair swap. Pure checkpoint reconciliation after repair divergence was rejected because it perturbed FP64 representative-gain arrays at the 1e-17–1e-16 level. On the common 8,192-candidate/six-family closure fixture, untouched 0.20.235a0 takes about 12.00 s while 0.20.236a0 takes about 11.02 s excluding persistence; including the one-time ~0.18 s authenticated cache write, the fresh chain is about 11.19 s. REPAIR itself improves from about 5.37 s to 4.27 s with exact selection/repair/qualification digests. Cumulative fresh-chain speedup versus the PERFBASE1-era 0.20.225a0 authority is about 2.44x. Remaining target-chain cost is dominated by the exact sequential sparse-state arithmetic itself, so no further CPU-only gate is justified under the exact-equivalence policy. `FINAL-GPU1` is next; positive accelerator qualification remains deferred to that workstation gate.

Part I - Foundations and ownership

Reader orientation

What an MLFF learns

An energy-conserving machine-learned force field represents a potential-energy function

$$E_\theta = E_\theta(\mathbf{Z}, \mathbf{R}, \mathbf{H}),$$

where $\mathbf{Z}$ contains atomic numbers, $\mathbf{R}$ contains positions, $\mathbf{H}$ is the periodic cell, and θ denotes model parameters. Forces and stress follow from derivatives of the same energy:

$$\mathbf{F}_i = -\frac{\partial E_\theta}{\partial \mathbf{R}_i}, \qquad \boldsymbol{\sigma} = -\frac{1}{V}\frac{\partial E_\theta}{\partial \boldsymbol{\epsilon}},$$

up to the exact stress sign and strain convention declared by the label source. MACE builds symmetry-aware local atomic features and sums atomic energy contributions [1]. A useful dataset therefore has to constrain both the energy surface and its derivatives throughout the intended simulation domain.

A low average force error is not sufficient. A model can fit common framework vibrations while failing on rare mobile-ion environments, strained cells, or migration geometries. Validation must include both numerical errors and physically relevant observables [6].

Why adjacent MD frames are not independent

A molecular-dynamics trajectory contains temporally correlated configurations. At a 1 fs output interval, neighboring frames are often nearly duplicates. Using them in different statistical roles creates leakage and overstates model accuracy.

For an observable x_t, the normalized autocorrelation at lag k is

$$\rho_x(k) = \frac{\langle (x_t - \bar{x})(x_{t+k} - \bar{x}) \rangle}{\langle (x_t - \bar{x})^2 \rangle}.$$

A truncated integrated autocorrelation time is

$$\tau_{\mathrm{int},x} = \Delta t \left[\frac{1}{2} + \sum_{k=1}^{k^\star} \rho_x(k) \right].$$

The effective number of independent observations is approximately

$$N_{\mathrm{eff},x} \approx \frac{T}{2\tau_{\mathrm{int},x}}.$$

Block averaging and hv-block cross-validation provide established foundations for handling correlated data [3-5]. The branch uses these ideas but records the mdstats-specific estimator, truncation rule, minimum block size, and purge rule as explicit policies.

The three ordinary dataset roles

Role	Function	May affect parameters?	May affect model choice?
Training	Supplies gradient updates	Yes	Yes
Validation	Early stopping and hyperparameter choice	No	Yes
Test	Final locked evaluation	No	No

MACE documents the same distinction: validation controls early stopping, while the test set is independent and evaluated at the end [8].

The architecture adds two more evidence roles:

Role	Function
Calibration	Calibrates committee disagreement or acquisition thresholds
Challenge test	Evaluates a named extrapolation or physical mechanism

Calibration is not test data. Challenge tests are not ordinary validation data.

Scope

Included

The branch will provide:

- VASP trajectory discovery and source certification;
- composition, temperature, ensemble, and strain reconstruction;
- electronic-structure compatibility and label-domain classification;
- energy, force, and stress label auditing;
- atomic-reference-energy identifiability diagnostics;
- frame-level eligibility and quality decisions;
- generic physical feature providers plus optional material-profile extensions;

LTA is an optional profile extension; it is not the generic feature or selection default.

- optional MPA-0 descriptors and zero-shot residuals;
- event detection before ordinary thinning;
- autocorrelation-aware complete-frame blocks;
- fixed outer validation, calibration, and locked test domains;
- independent cross-validation job families;
- fold-local transformations and training selection;
- deterministic nested training-size ladders;
- MACE target/replay artifact generation;
- replay-retention monitoring;
- training-only epoch resampling and exposure accounting;
- active-learning candidate screening, acquisition, and immutable lineage.

Excluded from the first runtime release

The first runtime sequence will not:

- patch the internal MACE optimizer or data loader;
- claim that coverage metrics prove final MLFF accuracy;
- infer an unstrained reference cell when more than one reference is defensible;
- merge incompatible DFT levels into one target head;
- use locked test labels for uncertainty calibration;
- treat replay-head disagreement as an uncertainty committee;
- silently download replay data from the mdstats core;
- promise efficient random access to XML before a streaming/indexed reader exists.

Reference application: bulk Li/Na/K-LTA

The first scientific target contains 27 AIMD runs:

- seven cation compositions: Li, Na, K, LiNa, NaK, LiK, and LiNaK;
- three temperatures: 300, 700, and 800 K;
- six additional LiNaK strain runs: hydrostatic $\pm 5\%$ volume, constant-volume orthorhombic $\pm 2\%$ linear strain, and engineering shear $\pm 2\%$;
- 1.4 ps per run at 1 fs time step;
- a Langevin NVT protocol, with approximately 0.2 ps initial relaxation.

This dataset motivates several domain-specific requirements:

1. Framework atoms greatly outnumber mobile cations. Global descriptor averages must not hide Li, Na, or K environments.
2. Strain combinations do not form a full Cartesian product with composition and temperature. Stratification must be hierarchical.
3. One trajectory per condition provides temporal interpolation evidence, not a fully independent replica test.
4. Fixed framework stoichiometry makes individual atomic reference-energy corrections non-identifiable without additional anchors.
5. Short trajectories may contain few cation hops. Absence of a transition is a documented coverage gap, not evidence that the transition is unimportant.

Relationship to existing mdstats capabilities

The training-data branch is an orchestrator over existing mdstats scientific capabilities.

Existing capability	Reused evidence
`mdstats.io.vasp.read_vasp_frames`	cells, coordinates, energies, forces, stress, temperature, provenance
`mdstats.io.vasp_controls.read_vasp_run_controls`	source controls, named energy channels, SCF iterations
VASP ensemble-control certification	NVE/NVT/NpT/NpH and driven-control classification
trajectory-quality assessment	source and trajectory integrity verdicts
production-regime assessment	transient and stationary regime evidence
Stage 11 structural modules	LTA rings, sites, coordination, topology, transitions
`mdstats.io.sampling_crossfit`	design precedent for source-bound blocks and purge semantics

The new branch owns dataset-level comparison, partition, selection, export, and active-learning lineage. It does not redefine the underlying physical analyses.

Controlling data flow

The controlling flow is:

```
source bytes
  -> source occurrence identity
  -> VASP controls + trajectory collection
  -> ensemble, quality, and production-regime evidence
  -> source catalog + decomposed label-domain audit
  -> structural atomic-reference identifiability
  -> immutable frame facts
       occurrence UID
       geometry fingerprint
```

```
      label payload digest
      labeled-configuration fingerprint
  -> labeled-frame eligibility
  -> full-resolution generic + partition-critical profile features
  -> event detection before ordinary thinning
  -> complete-frame temporal blocks
  -> fixed outer partition + PartitionIndependenceReport
      development pool
      outer monitor validation
      dedicated final-committee calibration cohort
      locked interpolation test
      named locked challenge tests
  -> independent cross-validation job family
      fold-training domain
      nested fold checkpoint monitor
      held-out evaluation fold
      fold-local feature metric + transform
      fold-local atomic-reference fit
      fold-local selection
      fresh model and frozen checkpoint per fold
      out-of-fold predictions
  -> final target-training transform + E0 fit + deterministic master order
  -> nested training-size ladder
  -> development MACE target/replay bundle
      no locked-test path
      replay-retention checkpoint constraint
  -> selected final checkpoints + independent-seed committee
  -> final-committee predictions on dedicated calibration cohort
  -> committee-bound uncertainty calibration
  -> post-freeze locked evaluation bundle
  -> active-learning candidate trajectories
  -> candidate admissibility + novelty + calibrated or rank-only uncertainty
  -> DFT query manifest
  -> labeled-round eligibility
  -> append-only child dataset generation with inherited roles
```

No arrow runs from a locked test into a fitted transform, E0 fit, hyperparameter or checkpoint choice, uncertainty calibration, or acquisition rule.

Package and ownership structure

```
mdstats/
  sampling/
    autocorrelation.py
    blocks.py
    assignment.py

  training_data/
    __init__.py
    policies.py
    records.py
    sources.py
    label_domains.py
```

```
    reference_energies.py
    conditions.py
    strain.py
    identity.py
    eligibility.py
    frame_catalog.py
    events.py
    independence.py
    partition_feasibility.py
    feature_metric.py
    blinding.py
    features/
      base.py
      thermodynamic.py
      geometry.py
      coordination.py
      lta.py
      mace.py
    partition.py
    cross_validation.py
    training_protocol.py
    objectives.py
    checkpoint_selection.py
    selection.py
    exposure.py
    replay.py
    replay_retention.py
    calibration.py
    active_learning.py
    role_inheritance.py
    export/
      extxyz.py
      mace.py
      manifest.py
    workflow.py
```

The proposed `mdstats.sampling` package contains source-independent primitives. Existing Stage 11 public records remain unchanged and may be reimplemented internally over these primitives only after exact replay tests pass.

Part II - Data and evidence contracts

Evidence records and immutability

Source facts

`TrainingDataSource` records facts derived from one source occurrence:

```
run_id
source_path
source_sha256
source_identity_signature
composition
```

```
timestep
frame_count
ensemble_certificate
quality_signature
production_regime_signature
label_domain_id
reference_group
```

Frame facts

`TrainingFrameRecord` contains only source-derived, policy-independent facts:

```
frame_uid
source_identity_signature
source_occurrence_signature
source_frame_index
time
atomic numbers
cell reference
label references
condition references
geometry_fingerprint
label_payload_digest
labeled_configuration_fingerprint
```

It does **not** contain eligibility, partition, selection, exposure, or acquisition state. The three fingerprints have different purposes and remain separate fields.

Decision records

Separate records are keyed by `frame_uid` or by an explicit dataset/job identity:

```
FrameEligibilityDecision
PartitionAssignment
SelectionAssignment
ExposureAssignment
CandidateAdmissibilityDecision
AcquisitionDecision
```

A new policy produces new decision records without mutating frame facts.

Workflow-policy and fitted records

The architecture also separates static policy templates from data-fitted or runtime-realized products:

```
PartitionRoleBudgetPolicy
PartitionFeasibilityReport
FeatureMetricPolicyTemplate
FoldFeatureMetricFit
FinalFeatureMetricFit
SelectionBudgetPolicy
TrainingObjectivePolicy
ConfigurationWeightPolicy
PropertyWeightPolicy
CheckpointMetricPolicy
TrainingProtocolIdentity
```

```
MaceCheckpointControlPolicy
ExposureBackendPolicy
MaceExposureRealizationRecord
ReplayRetentionPolicy
ProtocolFreezeRecord
CalibrationApplicabilityDomain
CalibrationTransferDecision
```

A policy template specifies an algorithm and fixed choices. A fitted record contains parameters learned from one declared training domain. A realization record contains behavior actually observed from an external tool. These roles are not interchangeable.

Content digests, not digital signatures

Previous documents used the word "signed" for deterministic record hashes. This manual uses precise terms:

- `content_digest`: canonical hash of one record;
- `policy_digest`: canonical hash of a policy;
- `source_digest`: hash of source bytes;
- `cryptographic_signature`: optional authenticated signature, not required in the first release.

A content digest detects modification but does not authenticate the author.

Source and manifest contract

Dataset manifest

A YAML or JSON manifest supplies source paths and information that cannot be reliably reconstructed from a single `vasprun.xml`:

```yaml
dataset_id: bulk-lta-initial
system_profile: lta

runs:
  - run_id: Li_300K
    vasprun: raw/Li_300K/vasprun.xml
    reference_group: Li_bulk
    replica_id: seed001

  - run_id: LiNaK_hydro_plus_005
    vasprun: raw/LiNaK_hydro_plus_005/vasprun.xml
    reference_group: LiNaK_bulk
    reference_run_id: LiNaK_700K
    assertions:
      intended_strain_class: hydrostatic
      intended_volume_change: 0.05
```

Manifest values are either:

- source locators;
- grouping declarations;
- scientific assertions to verify;
- explicit expert overrides with rationale.

A directory name is never treated as a physical label without verification.

Occurrence, geometry, and label identities

Source-occurrence identity

The DATA2 `source_identity_signature` is content-derived and may therefore be shared by byte-identical copied sources. DATA3 first binds that identity to one manifest occurrence:

$$\text{source\_occurrence\_signature} = \text{SHA256}(\text{run\_id}, \text{source\_locator}, \text{source\_identity\_signature}).$$

The frame occurrence is then

$$\text{frame\_uid} = \text{SHA256}(\text{source\_occurrence\_signature}, \text{source\_frame\_index}).$$

This identity is stable under later concatenation and export, while two copied sources declared as distinct manifest runs intentionally receive different occurrence identities.

Source-independent geometry fingerprint

`geometry_fingerprint` identifies the atomic geometry independently of labels. The first implementation supports exact copy/restart overlap detection using:

- ordered atomic numbers;
- canonical wrapped fractional coordinates;
- canonical cell representation;
- explicit numerical tolerances.

Energy and forces are deliberately excluded. The same geometry evaluated at a different DFT level or convergence threshold must still be detectable as the same geometry.

Label payload digest

`label_payload_digest` hashes the selected energy, forces, stress or virial, label-domain identity, and their declared numerical representation. It detects whether two occurrences carry the same labeled payload.

Labeled-configuration fingerprint

`labeled_configuration_fingerprint` combines the geometry fingerprint and label payload digest. It answers the narrower question: "is this the same geometry with the same labels?"

Leakage audits use all of the following:

- exact `frame_uid` overlap;
- exact geometry-fingerprint overlap;
- exact labeled-configuration overlap;
- near-geometry or descriptor distance;
- forbidden temporal proximity.

Later revisions may add permutation-, basis-, and symmetry-aware approximate geometry matching without changing these identity roles.

Electronic-structure label domains

Why the fingerprint is decomposed

Electronic-structure settings do not all have the same meaning. The architecture separates five records.

`TheoryIdentity`

Examples:

- exchange-correlation functional;
- DFT+U form and parameters;
- pseudopotential or PAW datasets;
- spin formalism;
- dispersion or hybrid-functional settings.

`EnergyReferenceIdentity`

Examples:

- energy channel;
- smearing/free-energy convention;
- atomic reference convention;
- per-cell versus per-atom normalization.

`DerivativeConvention`

Examples:

- force sign and units;
- stress versus virial;
- stress sign;
- tensor/Voigt representation;
- shear convention.

`NumericalQualityProfile`

Examples:

- ENCUT;
- k-point density;
- EDIFF;
- PREC;
- LREAL;
- LASPH;
- SCF iteration behavior.

`SoftwareProvenance`

Examples:

- VASP version;
- parser version;
- POTCAR hashes;
- source-control reconstruction version.

A versioned `LabelCompatibilityPolicy` determines whether differences are:

```
compatible
compatible_with_quality_flag
separate_label_domain
unresolved
```

Exact equality of the complete fingerprint is not required, but theory- or reference-defining differences cannot be waived silently.

First-release MACE rule

One MACE bundle contains exactly one target `LabelDomain` and optionally one foundation replay head. If the dataset contains two incompatible target DFT levels, mdstats produces two target bundles.

General arbitrary multi-target-head export is deferred until a later MACE adapter revision. This restriction makes the initial data contract unambiguous while preserving a path to MACE's general multi-head capability [11, 12].

Energy-channel policy

VASP forces and stress are derivatives of the electronic free-energy surface at the chosen electronic smearing. The selected `REF_energy` must therefore be an explicit named channel consistent with the derivative labels [7].

Example:

```
VaspEnergyLabelPolicy(
    channel="e_fr_energy",
    require_complete=True,
    derivative_consistency="electronic_free_energy",
)
```

The exact channel, units, completeness, and provenance are exported.

Atomic reference-energy audit and fitting

MACE commonly writes the total energy as

$$E = \sum_i E_{0,Z_i} + E_{\text{interaction}}.$$

When foundation-model corrections are estimated, one solves a system of the form

$$\mathbf{A}\,\Delta\mathbf{e}_0 \approx \mathbf{b},$$

where A_{cZ} is the count of element Z in configuration c, and b_c is the target-minus-foundation energy residual. Current MACE uses a least-squares solution, reports matrix rank, and warns when the element-count system is rank deficient [10].

The architecture separates two operations that have different leakage rules.

Structural identifiability audit

`AtomicReferenceIdentifiabilityReport` depends only on elemental count vectors and an atomic-reference policy. It may be created before partitioning and contains:

```
element order
count matrix shape
rank
singular values
condition number
null-space dimension
identifiable linear combinations
policy outcome
transfer limitations
```

It does **not** contain fitted elemental corrections or a fit residual.

Allowed structural outcomes are:

```
identified
rank_deficient_but_fixed_domain_usable
user_supplied
isolated_atom_anchored
foundation_preserved
rejected
```

For fixed-stoichiometry LTA, individual Si, Al, and O corrections are not all identifiable. A rank-deficient system may still be accepted for the same fixed stoichiometric manifold, but its null space and transfer restrictions must be explicit.

Training-domain atomic-reference fit

`AtomicReferenceFitRecord` is a fitted object. It contains:

```
training-domain frame UIDs
element support by element
structural-identifiability report digest
foundation-checkpoint digest
fitted elemental corrections
least-squares residual
solver and numerical tolerance
policy outcome
```

It may inspect only the applicable target-training domain:

- each cross-validation job has a separate fold-local fit;
- the final production run has a separate final-training fit;
- outer monitor, calibration, held-out fold, and locked-test labels are excluded.

Before fitting, every required element must have sufficient support in that training domain. Missing-element or newly rank-deficient fold fits fail or use an explicitly declared alternative such as user-supplied or foundation-preserved references.

`E0s: estimated` is emitted only when the corresponding training-domain fit is accepted. The bundle must state that rank-deficient offsets are not transferable to a different Si/Al ratio, defect count, cation count, salt phase, or interface.

Ensemble, temperature, and strain

Ensemble

The branch consumes the existing mdstats control certificate. It distinguishes at least:

```
NVE
NVT
NpT
NpH
temperature ramp
constant-velocity path
driven nonequilibrium
multi-thermostat
unresolved
```

Ensemble is not inferred merely from observed cell variation.

Temperature

A `TemperatureCondition` stores:

- requested `TEBEG` and `TEEND`;
- thermostat target;
- instantaneous ionic temperature series;
- production-regime mean and uncertainty;
- drift and stationarity diagnostics;
- ramp status.

Nominal and realized temperature remain separate.

Reference-cell resolution

Strain requires an explicit reference. The resolution order is:

1. explicit cell matrix;
2. explicit reference structure;
3. explicit reference run;
4. a unique compatible unstrained run in the same reference group;
5. unresolved.

Ambiguity fails closed.

Strain tensor

The cell-matrix convention is normative. ASE stores the three lattice vectors as **rows** of `Cell.array`. Fractional row vectors map to Cartesian row vectors as

$$\mathbf{r}_{\mathrm{row}} = \mathbf{s}_{\mathrm{row}}\mathbf{H}.$$

For reference cell $\mathbf{H}_0$ and current cell $\mathbf{H}_t$, the deformation gradient acting on Cartesian **column** vectors is

$$\mathbf{F}_t = \left(\mathbf{H}_0^{-1}\mathbf{H}_t\right)^T.$$

Equivalently, a row-vector implementation may use the right-acting map $\mathbf{H}_0^{-1}\mathbf{H}_t$ provided that all reported tensors are converted to the declared Cartesian-column convention before serialization. Use the right polar decomposition

$$\mathbf{F}_t = \mathbf{R}_t\mathbf{U}_t$$

to separate rotation from stretch. Record:

- volume ratio;
- linearized strain;
- Green-Lagrange strain;
- logarithmic strain;
- hydrostatic component;
- deviatoric norm;
- principal strains;
- tensor shear;
- engineering-shear equivalent;
- rotation;
- storage convention and coordinate frame.

Classifications are:

```
unstrained
hydrostatic
orthorhombic_or_deviatoric
shear
mixed_strain
variable_cell_fluctuation
unresolved
```

The fixture suite must include a nonsymmetric shear and a rotated stretch. Hydrostatic and diagonal fixtures alone cannot detect a transpose or left/right multiplication error.

Hierarchical condition schemas

The current LTA data do not occupy a full composition-temperature-strain Cartesian product. The LTA profile therefore defines:

```
unstrained family:
    composition x temperature x regime

strained family:
    composition x reference-condition x strain-mode x sign x regime
```

Only observed and scientifically applicable strata are required. Empty combinations are not treated as missing data.

Stress and virial contract

The MACE export specification is normative about stress.

Canonical `REF_stress` is:

- Cauchy stress in eV/Angstrom^3;

- a symmetric 3 x 3 tensor in Cartesian coordinates;
- ASE sign convention as verified against the chosen MACE release;
- no engineering factor applied to off-diagonal tensor components.

If six-component storage is used in an intermediate artifact, its order is explicitly recorded and round-tripped through ASE. Virial labels are stored under a different key and never silently relabeled as stress.

Every source domain must pass:

1. unit conversion test;
2. sign test against a finite strain energy derivative;
3. 3 x 3 to Voigt round trip;
4. shear-component test;
5. MACE read-back test.

Frames without valid stress may still train on energy and forces by using a zero stress weight, but only under an explicit heterogeneous-label policy [8].

Eligibility and quality screening

Run-level state

```
strictly_qualified
degraded_quality
unqualified
unresolved
```

The branch reuses the existing mdstats trajectory-quality evidence and records all overrides.

Labeled-frame eligibility

`FrameEligibilityDecision` applies after DFT labels exist.

Hard rejection includes:

- missing selected energy;
- missing or nonfinite forces;
- nonfinite positions or cell;
- singular cell;
- corrupt atom identity or ordering;
- truncated ionic step;
- catastrophic overlap;
- disallowed SCF nonconvergence.

Soft flags include:

- transient regime;
- high but physical force;
- unusual pressure or stress;
- rare coordination;
- cation transition;
- topology change;
- high foundation-model residual;

- degraded numerical quality.

A percentile tail alone is never a rejection reason.

Candidate admissibility before DFT

An active-learning candidate has no DFT labels. It receives a separate `CandidateAdmissibilityDecision` based on:

- finite cell and coordinates;
- minimum-distance safety;
- chemically allowed elements and counts;
- topology or framework sanity;
- integrator and trajectory integrity;
- model committee outputs;
- descriptor availability.

After DFT labeling, the candidate becomes a source occurrence and undergoes the full labeled-frame eligibility audit.

Material-profile and feature-provider architecture

DATA9A7a implements the declarative profile boundary:

```
class SystemProfileProvider(Protocol):
    provider_id: str
    provider_version: str

    def build_profile(self) -> MaterialProfileIdentity: ...
    def build_atom_groups(self, profile) -> AtomGroupCatalog: ...
    def build_condition_axes(self, profile) -> ConditionAxisCatalog: ...
    def build_independence_axes(self, profile) -> IndependenceAxisCatalog: ...
```

This first protocol deliberately does not calculate scientific features. It identifies the material, its phases, geometry, chemistry modifiers, optional extensions, meaningful atom groups, condition axes, and independence axes. The user explicitly declares the production profile; an automatic suggestion is advisory until confirmed.

A material profile is compositional rather than a flat enum. One or more phase components are combined with a geometry. For example, a solid-liquid interface contains separate crystalline-solid and liquid phase components under the `interface` geometry. Structural extensions such as `porous_network`, `zeolite`, and `lta` are optional and hierarchical. The generic one-phase fallback defines only `all_atoms`; a multi-phase profile must explicitly define phase membership.

DATA9A7b implements the first separate provider catalog for selection-grade local structure and generic structural events. Later stages add phase-specific activation, partition-critical profile features, additional selection evidence, and metric-group policies. They do not silently enlarge `SystemProfileProvider`, which remains the stable declarative identity contract.

The current DATA4-DATA7 implementation contains a generic universal path and optional provider-specific extensions. DATA9A7d migrates the LTA implementation behind the common extension envelope; LTA-named Python attributes remain only as compatibility views for historical bundles.

A representative solid-liquid interface profile is expressed as:

```
profile_id: lta-salt-interface
geometry: interface
phases:
  framework:
    phase_kind: crystalline_solid
    atom_groups: [framework_atoms]
    chemistry_modifiers: [ionic, covalent_network]
  molten_salt:
    phase_kind: liquid
    atom_groups: [salt_atoms]
    chemistry_modifiers: [ionic]
extensions: [porous_network, zeolite, lta]
```

This profile declares identity only. DATA9A7b provides a universal structural provider that may be activated for any declared material profile; DATA9A7c and DATA9A7d decide which phase-specific and optional-extension providers are added. Analysis-owned validation calls remain separate.

A separate fold transformation implements:

```
class FoldFeatureTransform(Protocol):
    def fit(self, training_frame_uids): ...
    def transform(self, frame_uids): ...
```

Raw scientific features and fitted statistical transforms are therefore not confused.

Universal structural selection features (DATA9A7b)

The universal structural provider is an interpretable complement to learned MACE descriptors. For pair separation r_{ij}, it defines a continuous chemistry-scaled weight from the sum of declared covalent radii. Smooth coordination is the weighted degree, while the support-neighbor count records how many pair weights exceed the numerical floor. Radial Gaussian projections summarize neighbor-shell occupancy without locating RDF peaks or minima. Weighted Legendre moments summarize neighbor-pair angles, and weighted spherical harmonics produce rotationally invariant q_4 and q_6 order parameters. A Gaussian local-density proxy and neighbor-species entropy provide packing and chemical-mixing information.

These quantities are selection descriptors, not replacements for analysis-owned RDF, integer coordination, angle-distribution, structure-factor, or topology results. Missing angular moments are represented by a zero fill plus an explicit mask. All minimum-image, switching, radial-width, and orientational-order parameters are part of immutable policy evidence.

Frame descriptors aggregate atomic features by declared atom groups and by each element present in the authorized DATA6 domain. The element schema is not constructed from locked-test geometry. Generic temporal events identify large changes in smooth coordination, support-neighbor count, local density, orientational order, or same-atom minimum-image displacement. They are candidate selection anchors only; physical interpretation remains profile-specific.

Raw thermodynamic features

- total and per-atom energy;
- composition-relative energy;

- RMS, maximum, and quantile force statistics;
- per-species force statistics;
- pressure and stress invariants;
- temperature;
- volume and density;
- SCF iteration statistics.

Raw cell and geometry features

- cell lengths and angles;
- strain invariants;
- pair-specific minimum distances;
- coordination histograms;
- bond-length moments;
- bond-angle moments;
- coordination anomalies.

Partition-critical system-profile features

Partitioning must know the rare categorical states that it promises to cover. DATA4 therefore exposes a lightweight, full-resolution system-profile layer before the outer partition is locked.

A general profile may provide categorical phase, environment, defect, molecular, region, or event states. For the optional LTA extension this layer provides, at minimum:

```
framework/mobile-species roles
coarse 4R/6R/8R site class when resolvable
on-center/off-center class
coordination-change flag
site-change flag
ring-crossing flag
framework-integrity flag
```

These features are designed for strata and event protection, not for final high-dimensional selection. If a required partition-critical classification is unresolved, the partition reports the missing coverage rather than claiming a balanced split.

Optional porous/zeolite/LTA extension

These features activate only when the declared material profile requests the corresponding extension. They are not defaults for ordinary crystals, liquids, amorphous systems, or interfaces.

- Si-O and Al-O coordination;
- tetrahedral distortion;
- framework topology state;
- Li-O, Na-O, and K-O coordination;
- nearest 4R, 6R, and 8R identity;
- ring-center displacement;
- signed ring-plane distance;
- off-center displacement;
- site assignment;
- entry, exit, transition, and ring-crossing events.

Optional MACE features

An optional `mdstats[mace]` provider may compute:

- foundation checkpoint identity and SHA-256;
- MACE version and source snapshot;
- invariant atomic descriptors;
- species-separated descriptor summaries;
- declared atom-group and species environment descriptors;
- zero-shot energy, force, and stress residuals.

MACE exposes learned descriptors for atomic environments [2]. PyTorch and MACE remain optional dependencies.

Label-derived difficulty-feature blinding

Descriptors depend only on geometry and a frozen model and may be computed for all domains. Foundation residuals require DFT labels and are therefore private to an authorized training domain:

```
TrainingDifficultyFeatureCatalog
    Residuals allowed for fold-training or final-training selection.

BlindedEvaluationPredictionCatalog
    Predictions stored without exposing residual-based selection features.
```

Outer monitor, calibration, held-out-fold, and locked-test residuals must not enter selection reports or feature fitting. Locked-test labels and residuals remain sealed until post-freeze evaluation. A policy violation is a hard leakage failure, even when the split itself is unchanged.

Fitted transforms and heterogeneous feature metric

Raw feature providers are partition-independent. Dataset-dependent operations are represented by distinct objects:

```
FeatureMetricPolicyTemplate
FoldFeatureTransform[k]
FoldFeatureMetricFit[k]
FinalFeatureTransform
FinalFeatureMetricFit
```

For fold k, fitting may inspect only the fold gradient-training domain. The held-out fold, fold checkpoint monitor, outer monitor, calibration cohort, and locked tests may be transformed using frozen parameters but cannot influence the fit.

A `FeatureMetricPolicyTemplate` defines:

```
raw feature blocks
per-feature robust scaling rule
per-block normalization
retained dimension or PCA rule
block weights
species weights
missing-block behavior
distance metric
dtype and numerical tolerance
```

A fitted metric records the medians, scales, projections, covariance factors, and retained dimensions learned from its declared training domain.

A block-normalized distance can be

$$d^2(i,j) = \sum_b w_b \, \| \, \mathbf{z}_i^{(b)} - \mathbf{z}_j^{(b)} \, \|_2^2 \, / \, d_b,$$

where b is a feature block, d_b is its retained dimension, and w_b is an explicit physical weight. Species-specific atomic-environment distances are reported separately from configuration-level distances. Dividing by retained dimension prevents a high-dimensional MACE descriptor block from dominating low-dimensional physical features solely through component count.

Fold-local and final transforms and metric fits are serialized separately from the static template.

Event detection before thinning

Rare events may be shorter than a preliminary stride. The controlling order is:

1. source and label integrity screening on all frames;
2. event and change-point detection on all eligible frames;
3. preservation of compact event windows;
4. temporal thinning of the ordinary non-event pool;
5. descriptor and physical-feature selection.

An event window may include one frame before, a representative event frame, and one frame after. The exact stencil is policy-controlled. Dozens of adjacent frames from one event are not retained unless required by a transition-path analysis.

Autocorrelation and complete-frame blocks

Observable families

Fast observables:

- potential energy;
- force RMS;
- pressure;
- temperature.

Slow observables:

- mobile-ion coordination;
- site identity;
- ring-plane coordinate;
- framework topology state.

Fast autocorrelation controls the minimum ordinary block size. Slow variables diagnose whether site-level independence is available at all.

Candidate stride in frames

The candidate stride is dimensionally defined as

$$s_{\mathrm{candidate}} = \max\left[s_{\mathrm{min}}, \left\lceil \frac{c\tau_{\mathrm{fast}}}{\Delta t_{\mathrm{frame}}} \right\rceil\right],$$

and

$$\Delta t_{\mathrm{candidate}} = s_{\mathrm{candidate}} \Delta t_{\mathrm{frame}}, \qquad 0 < c \leq 1.$$

The stride applies only to the non-event pool.

Complete-frame blocks

A `TrainingDataBlock` contains whole configurations over a contiguous interval:

```
block_id
run_id
frame_start
frame_stop
represented_time
regime
correlation diagnostics
configuration fingerprints
```

Atoms from one frame are never assigned to different partitions.

Purge width

A purge interval separates statistical roles. The policy records whether the purge is based on:

- a multiple of fast autocorrelation time;
- a minimum physical duration;
- event boundaries;
- restart-overlap detection.

If a slow state never decorrelates, the report states that blocked temporal splitting does not provide state-level independence.

Part III - Statistical design and selection

Outer partition architecture

Independence hierarchy

Use the strongest available evidence level:

1. independent replica or velocity seed;
2. independent cation ordering;
3. independent thermodynamic run;
4. purged temporal block within one run.

Temporal separation does not create an independent metastable state when the slow variable never decorrelates.

Partition-role feasibility

Before assigning roles, a `PartitionRoleBudgetPolicy` states the requested cohorts and minimum support. A `PartitionFeasibilityReport` evaluates whether the available independent blocks can support:

```
development_pool
outer_monitor_validation
uncertainty_calibration
locked_interpolation_test
locked_challenge_tests
cross-validation folds
nested checkpoint monitors
purge intervals
```

Possible outcomes are:

```
fully_supported
supported_with_temporal_blocks_only
calibration_deferred
challenge_set_external_only
reduced_cross_validation_folds
insufficient_for_locked_test
insufficient_for_requested_roles
```

The workflow never carves every desired role from a short trajectory merely to satisfy a percentage. A calibration cohort or challenge set may be deferred to later independent calculations.

Outer domains

For each target `LabelDomain`, define:

```
development_pool
outer_monitor_validation
uncertainty_calibration
locked_interpolation_test
zero or more locked_challenge_tests
```

Development pool

Only this domain supplies cross-validation fold-training and final target training candidates.

Outer monitor validation

This fixed, representative domain controls final-run monitoring, stopping, and checkpoint choice and never supplies gradients. It is not the locked test.

Uncertainty calibration

This domain is reserved for predictions from the actual final independent-seed committee. Out-of-fold predictions may diagnose ranking behavior, but they do not automatically calibrate numerical final-committee thresholds.

Locked interpolation test

This domain estimates unseen-frame performance within sampled conditions. It cannot affect hyper-parameters, selection, calibration, acquisition, stopping, checkpoint choice, or protocol design.

Locked challenge tests

Examples include:

- omitted temperature;
- omitted composition;
- omitted strain mode;
- independent structural or chemical realization;
- migration-coordinate calculations.

These remain separate named evidence cohorts.

Machine-readable independence evidence

Every outer, fold-evaluation, checkpoint-monitor, calibration, and test cohort receives one or more evidence grades:

```
independent_replica
independent_structural_realization
independent_thermodynamic_run
purged_temporal_block
slow_state_not_decorrelated
insufficient_independence
```

The report records purge width, autocorrelation evidence, duplicate checks, and known limitations. Metrics must carry these grades.

Independent cross-validation job families

Invalid design that is prohibited

The same continuously trained model must not train on fold F_1, later call F_1 validation, and report the result as out-of-fold evidence. Once a frame has contributed a gradient, it is no longer independent validation evidence for that model.

The held-out evaluation fold must also not control early stopping or checkpoint choice. Selecting the best checkpoint on the evaluation fold would bias the reported fold error.

Correct cross-validation

For K evaluation folds, create K independent jobs. For job k, partition the non-evaluation development data into:

```
fold_training_domain_k
fold_checkpoint_monitor_k
held_out_evaluation_fold_k
```

The default checkpoint monitor is a deterministic, purged nested split carved from the non-evaluation data. A versioned policy may instead use a declared fixed monitor cohort, but the held-out evaluation fold is never used for model selection.

Train a fresh model:

$$M_k = \mathrm{Train}\ \left(S_k\left[T_k(\mathcal{D}_{\mathrm{fold\ train},k})\right], \mathcal{D}_{\mathrm{checkpoint},k}\right),$$

where:

- T_k is fitted only on the fold-training domain;
- S_k selects only from that domain;
- the fold-local atomic-reference fit uses only that domain;
- the checkpoint monitor controls stopping and checkpoint choice but no gradients;
- M_k has an independent initialization, optimizer, and checkpoint lineage.

Only after checkpoint choice is frozen is M_k evaluated on the held-out fold $\mathcal{F}_k$. Combining these predictions gives a genuine out-of-fold catalog.

Cross-validation output

```
CrossValidationJobFamily
  evaluation-fold definitions
  fold-training domains
  fold checkpoint-monitor domains
  fold-local transforms and FeatureMetricPolicy records
  fold-local training selections
  fold-local AtomicReferenceFitRecord objects
  one development MACE bundle per fold
  fresh-seed and initialization contract
  held-out out-of-fold prediction catalog
  aggregate metrics with independence grades
```

Every fold uses the same `SelectionBudgetPolicy`. Equal nominal target sizes are used where feasible; mandatory-anchor differences and actual counts are reported. Hyperparameter comparisons use the same budget policy and coverage criteria, not an assumption that different folds contain identical selected frames.

Cross-validation selects policies and estimates development-domain performance. It is not implemented as a rotating epoch schedule.

Training-set selection

Selection runs only inside the applicable fold-training or final-training domain. A `SelectionBudgetPolicy` fixes requested sizes, mandatory-anchor requirements, evidence-class quotas, and deterministic interleaving.

Deterministic quota-interleaved master order

The selector first resolves mandatory coverage anchors. Remaining positions are filled by a deterministic interleaving schedule across evidence classes:

```
representative coverage
species-environment coverage
rare events
descriptor FPS
difficulty enrichment
```

The policy stores either explicit counts or fractions for every target size. A representative default may reserve, after mandatory anchors:

```
representative coverage     45%
species environments        20%
rare events                  15%
```

```
descriptor FPS              10%
difficulty enrichment       10%
```

These values are project policy, not universal constants. Deficits in one class are redistributed by a declared deterministic rule. This prevents later selectors from being starved when an earlier class consumes the size budget.

Near-duplicate pruning occurs during construction. The result is one ordered sequence

$$q_1, q_2, \ldots, q_N,$$

and requested datasets are prefixes:

$$\mathcal{T}_n = \{q_1, \ldots, q_n\}.$$

A requested size below the mandatory-anchor count fails explicitly.

Mandatory hierarchical quotas

The generic rule is that every observed, applicable combination of declared condition axes and protected event classes receives an auditable minimum coverage request. The axis catalog is profile-provided and may include composition, temperature, pressure, strain, phase, defect state, surface termination, interface registry, molecular conformer, or preparation history.

For the optional LTA profile:

```
unstrained: composition x temperature x regime
strained: composition x reference-condition x strain-mode x sign x regime
```

Only applicable observed strata are required.

Representative anchors

Representative anchors preserve dense equilibrium regions and expected production frequencies. Diversity-only sampling is insufficient because it may overweight feature-space boundaries.

Configuration-level FPS

Use the fitted heterogeneous feature metric. Deterministic farthest-point sampling selects

$$i^* = \arg\max_i \min_{j \in S} d(\mathbf{z}_i, \mathbf{z}_j),$$

with stable `frame_uid` tie-breaking. Pure FPS is not the complete selector.

Atom-group-specific environment selection

Run separate environment selection for every declared focus atom group. Groups may be defined by species, molecule, phase, spatial region, defect neighborhood, interface side, or profile-generated tags. Selecting an atomic environment adds its parent configuration. Abundant atom groups cannot determine the complete selection. The historical LTA implementation uses Li, Na, and K groups; these identities are not core defaults.

Rare-event anchors

Include a compact temporal stencil around profile-declared events. General defaults include coordination or neighbor changes, connectivity changes, large nonaffine displacements, local-density changes, phase/order changes, strain extrema, and high but physical restoring-force excursions. Site changes,

ring-plane crossings, pore-window events, adsorption/desorption, or interphase transfer activate only when their profile providers are present.

Difficulty enrichment

Within the training domain only, add a controlled quota of configurations with large foundation-model residuals, stratified by condition and species. These label-derived features remain blinded in evaluation domains.

Coverage diagnostics

Report by feature block, condition, and species:

- candidate-to-training nearest distance;
- selected-to-selected nearest distance;
- 90th and 95th percentile covering radius;
- physical-feature quantiles;
- event/state counts;
- redundancy fraction;
- budget realized by evidence class.

These metrics recommend a coverage-complete size. Learning curves remain necessary to establish model adequacy.

Training objective, weighting, and exposure

Membership, weighting, and exposure are different

Training-set membership says a frame may be used. Weighting says how strongly its labels affect the loss. Exposure says when and how often it is presented.

`TrainingObjectivePolicy` binds:

```
loss family
energy/force/stress global weights
head weights
normalization conventions
missing-label behavior
robust-loss settings
```

`ConfigurationWeightPolicy` binds condition-, regime-, event-, and quality-dependent configuration weights. `PropertyWeightPolicy` binds per-configuration energy, force, stress, or virial weights.

`ExposureAssignment` records:

```
frame_uid
head_id
eligible epochs
actual gradient exposures
configuration weight
energy/force/stress weights
sampling probability
random-seed lineage
```

Atom-group force imbalance

A configuration may contain many more force components from an abundant host group than from a scientifically critical minority group. Selection diversity does not remove this loss imbalance. The first adapter uses the standard MACE configuration/property-weight interface and therefore does not claim a general atomwise group-weighted loss. It must:

- report force metrics for all declared evaluation groups;
- impose profile-declared group, stress, and replay constraints during checkpoint selection;
- record any custom atomwise or auxiliary objective as a distinct protocol identity.

The historical LTA profile defines framework and Li/Na/K groups. Other systems may define defects, adsorbates, interface atoms, reactive centers, rare elements, or molecular subunits.

Exposure backends

`NATIVE_MACE_FIXED`
`CUSTOM_EPOCH_RESAMPLE`
`MULTI_JOB_RESAMPLE`
`FINAL_REFIT`

`NATIVE_MACE_FIXED`

All selected target and replay frames are present in fixed files. MACE shuffles the training loader reproducibly. This is the only backend supported by the first adapter.

`CUSTOM_EPOCH_RESAMPLE`

A custom MACE/PyTorch adapter rebuilds eligible data loaders at epoch boundaries. This requires runtime integration and is not deliverable by files alone.

`MULTI_JOB_RESAMPLE`

A deterministic sequence of restart jobs uses different fixed subsets. Its optimizer/checkpoint lineage is explicit and it is not equivalent to one native MACE run.

`FINAL_REFIT`

After protocol and epoch rules are frozen, all declared development data may be used. If outer validation is consumed, the final model loses that independent monitor and may be judged only on locked external evidence.

MACE exposure realization

`MaceExposureRealizationRecord` compares exported intent with the actual loader:

```
real_pt_data_ratio_threshold
pre-MACE target/replay counts
post-MACE effective target/replay counts
implicit duplication factor
expected and observed batches
configuration, energy, force, and stress exposures
```

MACE 0.3.16 can duplicate fine-tuning-head data when the target/replay ratio is below the MACE real-point data-ratio threshold [18]. The exact loader field is recorded in the exposure realization above. The first adapter disables this behavior where the locked CLI permits it; otherwise the duplication is

declared in `TrainingProtocolIdentity` and audited as realized exposure. Silent exposure changes are prohibited.

Cross-validation is a job family, not an epoch mode.

Part IV - Training and evaluation

Multi-head replay and training-protocol contract

Concept

Multi-head replay fine-tuning trains a shared MACE backbone on target data and a foundation replay dataset with separate output heads. The replay objective helps limit catastrophic forgetting while the target head adapts [11, 12].

`TrainingProtocolIdentity`

Every cross-validation family and final run is bound to one complete protocol:

```
foundation checkpoint and head
naive or multi-head mode
replay source, selection, and monitor
training objective and property weights
target/replay head weights
exposure backend and realized-balancing policy
checkpoint metric
MaceCheckpointControlPolicy
replay-retention policy
optimizer, scheduler, epoch cap, and seed policy
MACE adapter lock
```

Cross-validation results apply only to this identity. Hyperparameters selected under naive fine-tuning are not automatically valid for replay fine-tuning.

Separate lineages

Target and replay data retain separate:

```
source catalog
label domain
atomic-reference policy
selection plan
training weights
exposure accounting
validation or sentinel monitoring
```

Replay source modes

```
MP_SHORTCUT
EXTERNAL_TRUE_LABEL
EXTERNAL_PSEUDOLABEL
PRESELECTED
```

The mdstats core records a `ReplayPreparationPlan`; it does not download replay data. The optional MACE adapter may execute or print the official MACE selection command.

Replay-retention monitor and constraint

A training-only replay file is insufficient. The bundle also contains a disjoint `replay_monitor.xyz` or named `foundation_retention_suite`.

For true-label replay, it measures held-out DFT errors. For pseudo-label replay, it measures drift from the original foundation model on unseen sentinel configurations.

A `ReplayRetentionPolicy` defines:

```
retention metric
foundation or pre-fine-tuning baseline
tolerated degradation delta
aggregation across energy/force/stress
failure or override behavior
```

Checkpoint metric and constrained choice

A `CheckpointMetricPolicy` defines the target checkpoint objective and all constraints. It must include:

```
primary target scalar
energy/force/stress normalization
Li/Na/K species metrics
worst-condition metrics
rare-event metrics
replay-retention constraint
missing-label behavior
```

A typical rule is

$$\min_{c} L_{\text{target monitor}}(c)$$

subject to

$$L_{F,\text{Li/Na/K}}(c) \leq \boldsymbol{\delta}_F, \qquad \Delta L_{\text{replay monitor}}(c) \leq \delta_{\text{replay}}.$$

The exact metrics and thresholds are project policy and are serialized.

MACE checkpoint-control policy

MACE 0.3.16 evaluates all validation heads but uses the **last** validation head for learning-rate scheduling, patience, and native best-checkpoint decisions [17]. Its multi-head assembly places `pt_head` before target heads in the versioned source [18], but this ordering is an implementation detail that must be tested rather than assumed.

The initial adapter supports:

```
NATIVE_TARGET_LAST_WITH_EXTERNAL_CONSTRAINT_AUDIT
```

It must:

1. verify by source lock and smoke test that the target checkpoint monitor is the last validation head controlling native scheduling;
2. use a fixed epoch cap and configure patience so the run is not terminated by replay-head behavior;
3. enable retention of every evaluation checkpoint;

4. evaluate each candidate checkpoint externally on the target checkpoint monitor and replay monitor;
5. apply `CheckpointMetricPolicy` deterministically;
6. fail closed if the tested head-order or checkpoint behavior changes.

Later modes may provide full external scheduler control or a custom training loop. A post-training audit alone is insufficient if native early stopping was allowed to terminate on the wrong head.

Exposure diagnostic

A coarse intended ratio is

$$R_{\text{exposure}} = \frac{N_{\text{replay}} w_{\text{pt}}}{N_{\text{target}} w_{\text{target}}}.$$

The realized record additionally counts implicit duplication, batches, and energy/force/stress exposures. Intended counts never substitute for observed loader behavior.

MACE adapter and output contract

Version lock and compatibility matrix

The initial adapter targets `mace-torch==0.3.16`, the current PyPI release at this architecture revision [9]. Every supported version records:

```
mace version
package wheel/source SHA-256
Git commit or tag
mace_run_train --help
fine_tuning_select --help
key parser, loader, and train-loop source digests
validated head order
validated checkpoint-control behavior
validated replay-ratio behavior
```

Documentation URLs alone are not treated as a stable API contract.

Minimal XYZ plus complete sidecar manifest

Extended XYZ contains only MACE-readable labels, weights, and compact stable identities. Long provenance and reason lists live in a sidecar frame manifest keyed by `frame_uid`. DATA8 writes Cartesian positions and per-atom floating labels with 17 significant decimal digits rather than ASE 3.29's eight-decimal default, then certifies the artifact through a streamed ASE read-back.

Minimum target-frame XYZ fields are:

```
REF_energy
REF_forces
REF_stress
frame_uid
config_type
config_weight
config_energy_weight
config_forces_weight
config_stress_weight
```

The sidecar stores geometry/label fingerprints, source lineage, composition, temperature, ensemble, strain, regime, selection reasons, policy digests, and all audit evidence.

Separated development, calibration, and evaluation artifacts

```
mace_artifacts/
  development_bundle/
    target_train.xyz
    target_valid.xyz
    replay_train.xyz
    replay_monitor.xyz
    mace_config.yaml
    frame_manifest.json
    target_label_domain.json
    structural_atomic_reference_report.json
    atomic_reference_fit.json
    feature_metric_fit.json
    training_objective_policy.json
    checkpoint_metric_policy.json
    training_protocol_identity.json
    mace_checkpoint_control_policy.json
    replay_plan.json
    replay_retention_policy.json
    selection_manifest.json
    exposure_backend_policy.json
    adapter_lock.json
    cross_validation/
      fold_00/
        train.xyz
        checkpoint_monitor.xyz
        replay_train.xyz
        replay_monitor.xyz
        mace_config.yaml
        transform.json
        feature_metric_fit.json
        selection.json
        atomic_reference_fit.json
        training_protocol_identity.json
      fold_01/
        ...

  calibration_bundle/
    calibration.xyz
    committee_identity.json
    calibration_policy.json

  sealed_evaluation_bundle/
    target_test.xyz
    challenge_tests/
    evaluation_commands.yaml
    bundle_digest.json
```

```
  evaluation_activation/
    protocol_freeze_record.json
    selected_committee_identity.json
    activation_decision.json

  evaluation_results/
    evaluation_result_catalog.json
```

Replay files are omitted when replay is disabled. A sealed evaluation bundle may be prepared early, but it is not opened or referenced by training. Activation requires a `ProtocolFreezeRecord`, complete `TrainingProtocolIdentity`, selected committee digests, and checkpoint-selection decision.

Explicit E0 serialization

`AtomicReferenceFitRecord` is converted to the exact MACE input accepted by the version lock, normally an explicit atomic-number mapping:

```yaml
E0s:
  3:  -1.234
  8:  -2.345
  11: -3.456
  13: -4.567
  14: -5.678
  19: -6.789
```

The fit-record path and digest belong in provenance. A conceptual fit-record placeholder is never emitted as the MACE `E0s` value.

One target label domain per bundle

The development configuration contains one target head and an optional replay head. It contains no locked test path. Its exact schema is generated by the locked adapter and must preserve target-last validation control under the accepted checkpoint policy.

Export and loader round trip

The gate verifies:

1. ASE write/read equality;
2. atom order;
3. cell and PBC;
4. selected energy;
5. forces;
6. stress convention;
7. weights;
8. head labels and validation order;
9. explicit E0 mapping;
10. MACE parser recognition;
11. effective target/replay counts after loader assembly;
12. LAMMPS element mapping at later deployment.

Protocol-matched cross-validation and final training workflow

The recommended initial workflow is:

1. Build one immutable outer partition, feasibility report, and independence report.
2. Define candidate `TrainingProtocolIdentity` objects, including naive/replay mode, replay preparation, objective, exposure backend, and checkpoint policy.
3. For each protocol, create K independent jobs. Each has a fold-training domain, nested checkpoint monitor, held-out evaluation fold, and the same protocol-matched replay lineage.
4. Fit fold-local transforms, metric, selection, and atomic references using only each fold-training domain.
5. Train one fresh model per fold under the version-tested MACE checkpoint control. Freeze the externally audited checkpoint without inspecting the held-out evaluation fold.
6. Evaluate the frozen checkpoint on the held-out fold and collect out-of-fold predictions and independence grades.
7. Compare complete protocols using aggregate out-of-fold metrics and the fixed outer monitor. A naive protocol and a replay protocol are compared as different identities.
8. Freeze the selected data, objective, replay, exposure, stopping, checkpoint, and seed policies.
9. Fit final transforms, selection, and atomic references on the final target training domain.
10. Train independent final seeds under the same frozen protocol and record actual MACE exposure realization.
11. Apply constrained checkpoint selection and create the final committee.
12. Run that committee on the dedicated calibration cohort, record its applicability domain, and calibrate numerical uncertainty thresholds.
13. Create a `ProtocolFreezeRecord`; activate the sealed evaluation bundle and evaluate locked tests once.
14. Use the calibrated committee for active learning within its applicability domain; use rank-only acquisition outside it until recalibration.

If a final-refit mode consumes the outer monitor, its protocol must use a predeclared epoch/checkpoint rule and only locked external tests remain independent evidence.

Active-learning architecture

Immutable loop

```
trained independent-seed committee
  -> exploratory ASE/LAMMPS trajectories
  -> candidate occurrence catalog
  -> candidate admissibility
  -> physical events + descriptors + disagreement
  -> calibrated acquisition and burst deduplication
  -> DFT query manifest
  -> labeled source ingestion
  -> labeled-frame eligibility
  -> append-only child dataset version
  -> retraining
```

Acquisition evidence

A candidate may be selected using a Pareto or quota policy over:

- committee force disagreement;
- energy or stress disagreement;

- nearest-training descriptor distance;
- rare-event or physical-risk state;
- condition coverage gap;
- redundancy penalty.

A single weighted sum may be reported, but individual components remain available.

Calibration, committee binding, and applicability

Committee disagreement is a ranking signal, not an error guarantee [13, 14]. The architecture distinguishes:

```
OutOfFoldUncertaintyDiagnostic
    Tests whether uncertainty ranks error during development.

FinalCommitteeCalibration
    Sets numerical thresholds using predictions from the actual final
    committee on a dedicated calibration cohort.
```

A calibration record is bound to:

```
committee model digests
architecture and number of members
target-training lineage
replay lineage and retention policy
seed policy
MACE version and adapter lock
precision and inference settings
calibration-cohort identity
```

`CalibrationApplicabilityDomain` additionally records:

```
elements and compositions
temperature and strain range
cell-size range
site and event classes
descriptor-distance range
force/stress range
framework-integrity state
```

A `CalibrationTransferDecision` classifies each candidate domain as:

```
within_calibrated_domain
rank_only_outside_domain
recalibration_required
rejected_incompatible_domain
```

Out-of-fold predictions alone do not define the numerical scale for a committee trained on full development data. If no valid final-committee calibration cohort exists, the workflow emits only an explicitly **uncalibrated rank-only** acquisition plan.

Report:

- Spearman uncertainty-error correlation;
- high-error recall in top uncertainty quantiles;

- false-negative rate;
- per-species and per-condition calibration;
- applicability-domain coverage;
- calibration transfer warnings when committee identity or candidate domain changes.

Locked tests are excluded.

Burst deduplication

Adjacent uncertain frames from one event are clustered by trajectory, time, geometry fingerprint, descriptor distance, and event identity. A compact representative stencil is selected.

Append-only role inheritance

A child dataset inherits all existing frame roles unchanged by default:

```
existing development/validation/calibration/test roles
    -> inherited unchanged

selection-biased active-learning labels
    -> new development/training candidate pool

independent random labels from a newly entered domain
    -> possible new calibration or validation cohort

predeclared physical challenge calculations
    -> new named locked challenge set
```

A complete repartition is permitted only as a new evaluation lineage with a new partition identity. Its metrics must not be presented as directly comparable to the old locked-test lineage without qualification.

Determinism and reproducibility

Every build records:

- source digests and source identities;
- parser and mdstats versions;
- policies and policy digests;
- reference-cell identities and cell-matrix convention;
- feature-provider versions;
- foundation checkpoint digest;
- MACE adapter lock and compatibility-test evidence;
- random seeds and floating-point dtype;
- `FeatureMetricPolicyTemplate` plus fold/final fitted metrics;
- fold checkpoint-monitor policy;
- fold and final `AtomicReferenceFitRecord` objects;
- `PartitionRoleBudgetPolicy`, feasibility, and independence reports;
- `SelectionBudgetPolicy` and realized evidence-class budgets;
- `TrainingObjectivePolicy`, configuration/property weights, and `CheckpointMetricPolicy`;
- complete `TrainingProtocolIdentity`;
- MACE checkpoint-control and exposure-backend policies;

- `MaceExposureRealizationRecord`;
- replay-retention and checkpoint-selection decisions;
- protocol-freeze and evaluation-activation records;
- calibration applicability and transfer decisions;
- active-learning role-inheritance policy;
- tie-breaking rules, fold assignments, selection master order, and output checksums.

Performance and storage

The first implementation processes one trajectory at a time. It stores compact metadata and feature arrays, releases full trajectories, and uses one of two explicit export policies:

```
SEQUENTIAL_REPARSE
    Reparse each source sequentially and emit selected frames.

SELECTED_FRAME_CACHE
    Cache selected atomic arrays during the first pass after the selection is
    known through a second controlled source pass.
```

The architecture does not promise XML random access. A later indexed or streaming VASP reader may replace the second sequential parse without changing scientific contracts.

Failure semantics

The workflow fails closed when:

- source or label identity is unresolved;
- required labels are absent or nonfinite;
- incompatible label domains are mixed;
- strain requires an ambiguous reference cell or the cell convention is unclear;
- requested partition roles are statistically infeasible under the declared independence policy;
- locked or monitor labels reach a fitted transform, E0 fit, selector, difficulty feature, calibration, or acquisition operation;
- `E0s: estimated` is requested without an accepted training-domain atomic-reference fit or exact adapter serialization;
- a cross-validation held-out fold controls checkpoint selection;
- a cross-validation family is not bound to the same complete training protocol used for final training;
- the tested MACE validation-head order or native checkpoint behavior changes;
- native MACE silently changes target/replay exposure without an accepted realization record;
- a locked-test path appears in a development MACE configuration;
- no checkpoint satisfies mandatory target, focus-group, or replay-retention constraints;
- replay checkpoint and replay source are incompatible;
- dynamic epoch resampling is requested through a fixed-file-only adapter;
- calibrated candidate acquisition is attempted outside the calibration applicability domain without rank-only fallback or recalibration;
- active-learning child generation reassigns existing roles without a new evaluation lineage.

The workflow reports, rather than fabricates, absent profile-declared transition events, independent replicas, strain-composition combinations, calibration cohorts, or challenge sets.

Part V - Multi-view target-data architecture

Motivation

A target-data subset must cover several physically meaningful feature views simultaneously. Optimizing only an average distance or one descriptor can hide a severe deficit in another required view. The multi-view design therefore treats each required family as an explicit coverage constraint, diagnoses full-pool feasibility before subset optimization, and preserves exact nested prefixes so target-size learning comparisons are not confounded by resampling.

The architecture follows four rules:

1. feasibility precedes subset optimization;
2. hard coverage cannot be traded for aggregate utility;
3. redundancy is defined through **unique covered witness mass**, not merely local density;
4. the selector and the independent coverage verifier remain separate authorities.

Exact neighborhood graph

For feature family m, let $x_w^{(m)}$ be witness coordinates, $x_c^{(m)}$ candidate coordinates, D_m the frozen scaling transform, and $r_w^{(m)}$ the authoritative witness radius. Define the exact binary adjacency

$$A_{wc}^{(m)} = \mathbf{1}\left[\left\| D_m\left(x_w^{(m)} - x_c^{(m)}\right) \right\|_2 \le r_w^{(m)}\right].$$

The production search is exact (`eps=0`) and uses `scipy.spatial.cKDTree` radius queries; SciPy exposes explicit worker control for these searches [26, 33]. Approximate-neighbor methods are outside the current scientific authority.

For a selected subset S, witness multiplicity and weighted family coverage are

$$n_w^{(m)}(S) = \sum_{c \in S} A_{wc}^{(m)},$$

$$C_m(S) = \frac{\sum_w \omega_w^{(m)} \mathbf{1}[n_w^{(m)}(S) > 0]}{\sum_w \omega_w^{(m)}}.$$

With frozen hard threshold $\tau = 0.95$, the robust deficit is

$$D_{\max}(S) = \max_m \max(0, \tau - C_m(S)).$$

A weighted average is not a substitute for this worst-view condition.

FEAS1 - feasibility, fragility, and capacity evidence

FEAS1 evaluates the complete eligible development pool before subset optimization. It verifies expected self-cover, measures cross-support fragility, records candidate-degree histograms, and derives conservative lower bounds on the cardinality required to satisfy hard support/obligation constraints.

For witness w, full-pool support degree is

$$d_w^{(m)} = \sum_{c \in \mathcal{C}} A_{wc}^{(m)}.$$

Low-degree witness mass identifies fragile regions where deletion or correlation-unit exclusion can destroy support. FEAS1 may diagnose `cross_support_fragile` without changing the frozen hard threshold. A proven lower bound above the fixed 16,384 ceiling is a capacity diagnosis, not permission to relax coverage.

MVIDX1 - one shared sparse graph, not a second neighborhood search

MVIDX1 SHALL reuse the exact neighborhood output produced by FEAS1 whenever the semantic identity matches. FEAS1 and MVIDX1 are therefore consumers of one internal **ExactNeighborhoodEngine**, not separate geometric implementations.

The canonical execution substrate for each family is witness-oriented CSR-equivalent storage:

- `witness_offsets`: 64-bit offsets when required by edge count;
- `candidate_indices`: `uint32` when candidate cardinality permits;
- FP64 scientific weights stored separately;
- content identity bound to domain/candidate ordering, family/scaling identity, witness coordinates, radii, distance semantics, and cache-format version.

Worker count, query block size, queue depth, and other execution-only knobs SHALL NOT enter the scientific neighborhood identity. Changing parallelism must not invalidate an exact cache.

CSR/CSC compressed sparse representations store one contiguous index array plus pointer offsets; SciPy documents the canonical row/column forms and conversions [34]. MVIDX1 adopts the authenticated FEAS1 witness-to-candidate CSR and constructs the candidate-to-witness inverse graph without repeating cKDTree geometry.

Stable parallel CSR-to-CSC transpose

The inverse graph is constructed by a deterministic two-pass algorithm:

1. parallel block-local candidate-degree histograms;
2. canonical prefix reduction to global candidate offsets;
3. precomputed deterministic destination ranges per block;
4. parallel fill into disjoint ranges without atomics;
5. verification that forward and inverse edge counts and identities agree exactly.

This exposes parallelism while preserving canonical within-candidate witness order.

MVSEL1 - deterministic progressive selection

Selection constructs one global order whose prefixes are the planned target sizes. Phase A services mandatory reservations and unsatisfied hard views/strata. Phase B fills remaining capacity with a density-aware representative objective after hard obligations are met.

At each rank, admissible candidates are compared lexicographically by frozen priorities including worst-view deficit reduction, newly covered weighted mass, provenance/correlation balance, representative gain, normalized diversity, and stable frame identity. Rank generation is sequential because selection state changes after every accepted candidate; exact performance work therefore targets sparse incremental state updates rather than speculative rank selection.

The selector maintains per-witness multiplicity and per-candidate marginal state. When a witness changes state, inverse adjacency updates only candidates touching that witness. Full candidate-by-witness rescoring after every rank is forbidden.

REPAIR1 - exact shell repair from multiplicity

For selected candidate c, exact unique covered mass is obtained from witnesses with multiplicity one:

$$U(c \mid S) = \sum_{m} \sum_{w : A^{(m)}_{wc} = 1} \omega^{(m)}_w \, \mathbf{1}[n^{(m)}_w(S) = 1].$$

This avoids literal leave-one-out recomputation of complete coverage. Removal candidates must have negligible unique contribution and no unique hard/provenance role. Replacement candidates are drawn from the deficit frontier and every accepted swap must strictly improve the frozen lexicographic objective while remaining inside the active shell; lower-rung prefixes never change.

Within one repair iteration, proposal evaluations share an immutable pre-swap state and may execute concurrently. The accepted proposal is chosen afterward by the original deterministic comparison order.

MVQUAL1 and independent authority

MVQUAL1 compares legacy and multi-view subsets at identical cardinality using independent coverage recomputation. It records $D_{\max}$, aggregate deficit, uncovered mass/count, redundancy metrics, provenance/correlation diversity, and

$$N_{95} = \min\{N : \text{all hard predicates pass at size } \ N\}.$$

Selector-internal coverage is not accepted as independent qualification evidence. Locked-test data cannot tune radii, weights, repair budgets, or tie rules.

Fixed size/fidelity funnel

The planned nested rungs are

$$128, 256, 512, 1024, 2048, 4096, 8192, 16384.$$

Only hard-coverage-qualified rungs can survive the learning funnel. Candidate counts reduce as

$$8 \overset{\text{3 epochs}}{\rightarrow} 4 \overset{\text{10 epochs}}{\rightarrow} 2 \overset{\text{30 epochs}}{\rightarrow} 1.$$

The arrows denote surviving candidate count, not dataset-size halving. Fewer than four hard-qualified rungs fails closed before the 10-epoch stage.

Part VI - Performance and execution architecture

Performance objective

Optimization is accepted only when it preserves scientific authority and improves measured throughput, memory behavior, or restart cost. CPU percentage is diagnostic rather than the objective. In particular, memory-bound sparse kernels may be optimal below the nominal CPU occupancy target.

For a stage allocated P CPU lanes, define effective occupancy over a bulk interval as

$$U_P = \frac{\Delta t_{\mathrm{CPU}}}{P\,\Delta t_{\mathrm{wall}}}.$$

When at least $2P$ independent compute tasks are ready and the kernel is compute-bound, the target is sustained $U_P \gtrsim 0.85$ with automatic resource use capped by the configured campaign CPU fraction (90% by default). Throughput and wall time decide between exact-equivalent implementations.

Work/span model and global scheduling

The campaign adopts a task-parallel work/span view. Let T_1 be serial work and T_∞ the critical path. Ideal scheduling cannot beat

$$T_P \geq \max\left(\frac{T_1}{P}, T_\infty\right).$$

Classical work-stealing analysis motivates exposing many independent tasks to a common scheduler; for structured computations, the expected execution bound has the form $T_1/P + O(T_\infty)$ [32]. mdstats does not require a literal Cilk runtime, but adopts the same **work-conserving principle**: idle lanes take ready work from any compatible family/profile/domain instead of waiting for a local loop to finish.

Single-level parallelism

The default CPU realization SHALL expose parallelism at the highest level that provides enough independent tasks. Nested numerical parallelism is suppressed while the outer queue is populated:

$$P_{\mathrm{outer}} \times P_{\mathrm{native}} \leq P_{\mathrm{budget}},$$

with $P_{\mathrm{native}} = 1$ for cKDTree/BLAS/OpenMP calls when outer parallelism can fill the budget. This avoids oversubscription and the underfilled `one Python driver + briefly threaded native call` pattern observed before FEAS1-PERF3.

Libraries such as `threadpoolctl` can limit BLAS/OpenMP pools, but their controls are process-global and have caveats when manipulated from several Python threads [35]. The scheduler therefore treats native-thread configuration as a stage/resource-scope concern, not something each arbitrary worker toggles independently.

PARCORE1 - shared deterministic scheduler

`PARCORE1` is implemented in `mdstats 0.20.226a0`. The reusable queue class is `DeterministicWorkQueue`; it is now the common substrate for CPU-heavy independent work. Its execution contract provides:

- `StageResourceScope` CPU and RAM integration;
- separately bounded ready, submitted/in-flight, and completed queues;
- work-conserving dispatch across profiles/families/domains;
- deterministic ordered reducers for FP-sensitive authorities;
- native-thread quarantine when the caller supplies the explicit campaign resource scope;
- exception propagation with deterministic task identity;
- memory-weighted admission/backpressure plus explicit persistent-memory reservations;

- progress/heartbeat snapshots including ready, in-flight, completed, busy-lane, memory, and back-pressure state;
- locality/NUMA metadata that does not enter scientific identity.

The executor owns exactly `StageResourceScope.python_workers` executing threads. The queue MAY keep more submitted futures than executing lanes (the current FEAS1 realization permits up to twice the worker count) so an idle worker can immediately pull the next admitted task without waiting for coordinator hand-off. This does not increase the number of simultaneously executing Python lanes and does not authorize nested native parallelism.

Task completion may be out of order; `DeterministicOrderedReducer` commits authoritative FP64 reduction only in the prescribed canonical order whenever arithmetic order is part of exact-equivalence authority. FEAS1 is the first migrated consumer and retains its historical witness-block commit order exactly. Its parallel cKDTree tasks continue to use one native tree worker while the outer queue is populated.

Campaign execution passes an explicit `StageResourceScope`, so native BLAS/OpenMP limits are applied once by the queue-owning coordinator rather than toggled inside workers. Bare library/API calls that do not supply an explicit scope preserve their historical resource-control semantics; the scientific output is identical in either realization. `StageResourceScope.ram_budget_bytes` is execution-only and feeds queue admission.

Locality keys are stored now so future NUMA-aware scheduling can reuse the task model. PARCORE1 does **not** activate NUMA affinity or node-local stealing; those remain measurement-gated execution extensions.

NEIGHBOR1 - exact neighborhood production and reuse

`NEIGHBOR1` is implemented in `mdstats 0.20.227a0`. `ExactNeighborhoodEngine` is the single exact TARGET-DATA2B/C geometric implementation. Query blocks from all eligible families enter the PARCORE1 queue; while outer work is available, every cKDTree task uses one native worker [33]. The frozen scaled-Euclidean radius/tolerance semantics and candidate-frame deduplication order are unchanged.

As soon as a completed block becomes reducible in canonical witness order, FEAS1 now:

1. applies the historical support/capacity reduction in the same FP64 order;
2. appends the same exact row relation to a disk-backed canonical witness-oriented CSR stream;
3. releases the ragged temporary neighbor object.

Thus peak ragged-neighborhood memory scales approximately with active/buffered blocks rather than the complete family:

$$M_{\mathrm{ragged}} = O(PB\bar{d}),$$

where B is query-block size and $\bar{d}$ is mean neighborhood degree. The final CSR uses `uint64` witness offsets and `uint32` candidate indices. Its exact final allocation is known from streamed row counts/edge count and is admitted against `StageResourceScope.ram_budget_bytes` **before** materialization into RAM.

The cache is reconstructible execution state rather than a new scientific authority. Family identity binds label-domain ID, frame-domain/candidate ordering digest, family digest, candidate/witness cardinalities, frozen metric/tolerance semantics, and cache-format version. Worker count, query-block size, queue depth, timing, and progress settings are deliberately excluded. Native persistence authenticates manifests and each NumPy array by checksum plus scientific array reference; campaign storage records the cache independently of FEAS1 so restart may validate/reuse it.

MVIDX1 adopts authenticated forward CSR directly and performs no geometric query on a cache hit. `target_coverage_sparse_index.py` no longer owns a cKDTree/query-ball implementation. If the cache is missing, corrupt, or stale, MVIDX1 rebuilds forward CSR once through the same global `ExactNeighborhoodEngine`, persists it, and then proceeds. It must never revive the former duplicate serial-family/nested-tree geometry sweep.

`MVIDX-REUSE1` (`0.20.228a0`) parallelizes the remaining inverse/metadata work at the natural independent-component boundary. Required-family inversions and hard-obligation inversion are immutable tasks on `DeterministicWorkQueue`; each task uses the deterministic compiled SciPy CSR-to-CSC counting transpose with one native lane, and canonical required-family order is restored after completion. This avoids nested sparse-kernel parallelism and atomics while allowing the outer queue to occupy the campaign CPU budget. An experimental Python-threaded intra-family degree/prefix/range-fill realization was exact but slower on the frozen authority and was therefore rejected rather than promoted.

The prior row-by-row strict-order validator was also a measured MVIDX hotspot. Revision 95 replaces it with one vectorized adjacent-index comparison and masks pairs crossing CSR row boundaries. The predicate is mathematically identical: every within-row adjacent pair must remain strictly increasing; worker count and queue completion order remain execution-only.

COVREF-PAR1 - exact reference-radius block scheduling

`COVREF-PAR1` (`0.20.229a0`) removes the remaining one-driver/native-tree pattern from TARGET-DATA2B reference-radius construction. Each family still computes the identical robust scales, scaled coordinates, balanced reference masses, and exact leave-one-out local radius. The scaled matrix and one read-only `cKDTree` are constructed once per family; independent row blocks are then submitted to the stage-wide `DeterministicWorkQueue`, and every task calls the tree with `workers=1`. Results write to disjoint canonical row slices, so task completion order cannot alter the local-radius array. Direct API calls that omit `execution_scope` retain the historical native-tree `query_workers` realization for compatibility and oracle comparison.

Parallel block size is execution-only. The configured `radius_block_size` remains an upper bound, while the queue may reduce it to keep the estimated cKDTree temporary working set near 2 MiB and expose at least four blocks per assigned lane on sufficiently large families. This is especially important for 30k-40k-frame target domains: a fixed 1024-row block would expose only about 36 tasks and develop a large tail on a 28-lane workstation. Block boundaries are not scientific inputs; qualification requires byte-identical radii across block/worker schedules.

The family-adaptation path is also hardened without changing inclusion rules. Pair-geometry records are indexed once by `(frame_uid, rule_id)`, foundation species residuals once by `(frame_uid, atomic_number)`, and target-label scalar channels execute the exact historical `np.allclose` constant-family rejection before robust-statistic/tree work instead of after it. Weight-profile caching remains

content-derived execution state, and scaling is materialized once per family and shared read-only by radius tasks.

The campaign resolves TARGET-DATA2B construction workers separately from later coverage-scoring native-tree widths. Automatic COVREF uses the complete configured CPU budget as outer lanes; `StageResourceScope` fixes `tree_workers=1` and `blas_threads=1`, making $P_{\text{outer}} \times P_{\text{native}} = P_{\text{outer}}$ and preventing nested oversubscription.

Direct FEAS1, NEIGHBOR1-rebuild, and MVIDX inversion API calls that do not supply a `StageResourceScope` likewise retain historical host-independent execution semantics: their implicit scopes do not synthesize hard RAM ceilings from momentary shared-host/cgroup free-memory readings. Campaign execution, which owns the resource contract, continues to pass explicit RAM-bounded scopes and therefore remains fail-closed under declared memory limits. This distinction prevents transient unrelated host load from turning an otherwise identical direct scientific call into a scheduler-admission failure.

Memory budget and persistence

CPU admission is necessary but insufficient. The scheduler SHALL track an estimated memory budget

$$M_{\text{stage}} = M_{\text{trees}} + M_{\text{scaled}} + M_{\text{inflight}} + M_{\text{buffered}} + M_{\text{sparse}} + M_{\text{scratch}}.$$

New work is admitted only if both CPU and memory budgets permit it. Completed sparse blocks may spill to mmap-compatible uncompressed arrays so neighborhood reuse does not require retaining the complete graph in Python objects. Compression is optional and must be benchmarked because decompression can erase the saved neighborhood-search time.

NUMA-ready locality

A flat work queue is appropriate for the single-socket workstation but can inflate work on multi-socket EPYC/HPC nodes through remote-memory traffic and cache loss. NUMA-aware task runtimes explicitly address this locality problem [36]. PARCORE1 therefore reserves a locality extension:

- node-local queues and data shards;
- worker affinity to the owning NUMA node;
- local stealing first;
- cross-node stealing only to avoid idle lanes.

NUMA mode is execution-only and will be activated only after measured qualification on a suitable host.

Vectorization and exact numerical kernels

The optimization program prefers array kernels that replace repeated Python object traversal without changing arithmetic authority.

Ragged sparse gather

MVSEL/MVQUAL SHALL replace Python `list-of-slices -> concatenate -> repeat` patterns with offset-derived vectorized CSR gathers. Candidate/witness ordering is retained exactly.

Bounded-integer reductions
Species IDs, candidate IDs, and other bounded non-negative integer labels SHOULD use direct indexed reductions such as `numpy.bincount` when the semantic operation is counting or weighted summation [37]. This replaces repeated `unique + boolean mask` scans in FOUNDATION-AUDIT1, EVAL2, and sparse telemetry.

Stamp-array membership
REPAIR1 repeated set intersections SHOULD use epoch/stamp arrays for bounded witness IDs:

$$\mathrm{overlap}(j) = \mathbf{1}[\mathrm{stamp}[j] = e],$$

REPAIR-PAR1 realized proposal kernel
`REPAIR-PAR1` retains the sequential repair iteration and canonical winner authority. For each immutable iteration state, fused removal metrics scan each sparse family once; replacement frontiers are scored with canonical ragged-CSR gathers and thread-private epoch/stamp arrays rather than repeated `intersect1d`/`isin` calls. An inverse candidate-rank map replaces repeated linear future-rank searches. Proposal tasks may execute through PARCORE1 only when an execution-only sparse-edge work estimate exceeds the measured break-even threshold; smaller rungs remain serial. Completion order is never authoritative: proposal results are reduced in the historical removal-shortlist order using the unchanged objective/tie hierarchy. The selected replacement's representative contribution is recomputed by the historical scalar arithmetic before persistence, preserving the complete repair trace exactly. Worker count, adaptive threshold, queue depth, and stamp epochs are execution state and do not enter content identity.

where the current removed-witness set is stamped with epoch e. This turns repeated sorting/intersection work into direct indexed gathers.

Batched resampling/statistics
Independent bootstrap replicates and repeated quantiles SHOULD be processed in bounded vectorized batches. Static composition/species codes and other invariant indexing metadata are computed once per dataset/checkpoint domain and reused.

`AUDIT-EVAL-PERF1` applies this contract to EVAL2 with an execution-only bounded metadata cache keyed by the in-memory immutable evaluation view plus ordered correlation-block IDs. The cache stores precomputed composition keys, per-frame species membership, focus masks, and block codes; it does not enter target-role or prediction scientific identity. Paired bootstrap preserves the exact seeded draw stream while grouping draws into a temporary-memory-bounded matrix batch. FOUNDATION-AUDIT1 similarly shares one immutable DATA3 frame index and per-run species-membership map across audit domains and continues to read/authenticate the existing prediction sweep rather than invoking the foundation model.

Lookup and allocation hygiene
Hot loops SHALL avoid repeated linear `next(...)` searches, rebuilding immutable dictionaries/maps, repeated full-array scaling, unnecessary `concatenate`, and materializing Python objects when contiguous typed arrays suffice. Optimization reviews explicitly look for these patterns.

Stage-specific optimization map

Stage	Dominant issue	Planned exact optimization
TARGET-DATA2B reference-radius/coverage	serial block driver with nested cKDTree workers	global block tasks, one tree worker/task, early constant-family rejection, shared scaled workspaces
FEAS1	global PARCORE1 queue plus implemented streamed exact CSR	retain exact reduction; downstream sparse-kernel work only
MVIDX1	authenticated graph adoption; inverse/validator Python overhead	implemented MVIDX-REUSE1 component-level queue plus vectorized CSR validation
MVSEL1	Python ragged gathers and sparse update overhead	vectorized CSR gather, indexed weights, incremental counters; preserve sequential rank authority
REPAIR1	serial proposal shortlist and repeated intersections	implemented adaptive immutable-state proposal queue, vectorized frontier scoring, stamp arrays, fused sparse scans, O(1) rank map
MVQUAL1	independent same-N rescoring globally queued	PARCORE1 same-N jobs + batched sparse telemetry; canonical post-queue reduction
FOUNDATION-AUDIT1	per-frame/species Python reductions	implemented shared frame/species metadata, reused squared-error work, batched tail quantiles; no new inference
EVAL2 CPU analysis	repeated species/composition/focus reconstruction and bootstrap Python loops	implemented cached static reduction metadata, preallocated tails, memory-bounded batched bootstrap
REPLAY-UNIFY1	repeated serial ExtXYZ parsing/materialization	implemented source-SHA-bound byte-offset/natoms index; direct sparse seeks; deterministic bounded chunk parsing

Existing DATA6 GPU inference, training orchestration, structural FPS/GEMM kernels, and independent trajectory verification are not rewritten merely to increase thread count; they are changed only if runtime profiling identifies a new dominant hotspot.

REPLAY-PERF1 indexed replay realization

The selected replay ExtXYZ remains the only external replay authority. `ReplaySourceIndex` is reconstructible execution state keyed by the exact source bytes and source-artifact/source-order identities; it is never a substitute scientific authority. The index records frame byte offsets, byte lengths, and atom counts, allowing a requested subset such as the 2,000-frame monitor role to seek directly to those source frames. For a complete source traversal, contiguous frames are parsed in bounded chunks and their already-authenticated source-order geometry identities are reused. Parser chunk size is execution state and MUST NOT enter replay source, split, label, prediction, or view content identity.

ASE ExtXYZ parsing remains serial. REPLAY-PERF1 qualification explicitly tested thread-parallel chunk parsing and found it slower on the available CPU, so concurrency is not introduced merely to increase worker count. Future campaign-level parallelism MAY overlap independent higher-level consumers only if a later profile shows a net gain without changing persisted replay bytes or prediction authority.

CAMPAIGN-PERF-QUAL1 integrated reprofile and shifted bottleneck

The revision-102 closure profile validates cumulative behavior instead of inferring campaign speed from isolated kernel benchmarks. On a common 8,192-candidate/six-family target-data chain, untouched `0.20.225a0` completes FEAS1 -> MVIDX1 -> MVSEL1 -> REPAIR1 -> MVQUAL1 in about 27.26 s. The optimized realization through `0.20.234a0` completes the same scientific chain in a four-lane median of about 11.95 s (~2.28x faster) with exact output digests. Current one/two/four-lane wall times are about 12.91/12.07/11.95 s, so additional outer lanes no longer provide material end-to-end scaling on the qualification CPU.

The remaining four-lane wall-time composition is approximately 45% REPAIR1, 41% MVSEL1, 9% FEAS1 plus neighborhood production, 4% MVQUAL1, and less than 1% MVIDX1. Profiling shows the selector rank-choice routine itself is no longer dominant: 4,096 `_choose_candidate` calls consume about 0.90 s cumulative, while 4,096 exact `_select_and_update` calls consume about 5.20 s, including about 4.57 s in ordered paired sparse decrements.

REPAIR1 exposes a stronger exact-reuse opportunity. On the same fixture it executes about 4,098 additional `_select_and_update` calls (about 5.34 s cumulative in the profile) to reconstruct the already-known selector sparse state before and during repair preparation. Proposal work is materially smaller. This reconstruction is not a new scientific decision; it is deterministic state derivable from the MVIDX authority plus the ordered MVSEL selection. `MVSTATE-REUSE1` therefore becomes the next exact-equivalence gate: persist/authenticate enough terminal selector execution state for REPAIR1 to start from that state directly, while retaining the historical replay path as the qualification oracle.

The closure accepts a modest representative peak-RSS increase (about 306 MiB -> 343 MiB) because it is caused by reusable authenticated sparse execution state, remains far below the explicit campaign budget, and produces no observed queue backpressure. Performance evidence never overrides scientific digests.

MVSTATE-REUSE1 exact selector-state handoff and CPU closure

Revision 103 implements an authenticated `TargetMultiViewSelectionStateCache` at the MVSEL/REPAIR boundary. MVSEL snapshots the exact mutable selector state at every materializable rung. REPAIR may restore such a checkpoint only while repair has not diverged from the pure selector order. After the first accepted repair swap, the historical mutable repair state is carried forward. This restriction is normative: reconstructing a later repaired state from a pure MVSEL checkpoint plus selected-set differences changed FP64 representative-gain entries by about `1e-17`–`1e-16`, so that shortcut is rejected even though selected IDs were identical.

The cache is reconstructible execution state. Identity binds the reference/MVIDX/MVSEL/policy/sparse-kernel lineage and excludes worker/storage choices. Persistence uses one authenticated uncompressed NPZ array bundle plus a canonical manifest. A fresh campaign passes the in-memory cache directly from MVSEL to REPAIR and persists the same cache for restart; stale, missing, corrupt, or incompatible state falls back to exact historical replay. Post-divergence predetermined additions may batch CSR gather preparation, but candidate-major FP64 mutations remain in the historical order and are state-array qualified.

On the 8,192-candidate/six-family integrated fixture, untouched 0.20.235a0 has a target-chain median near 12.00 s and REPAIR near 5.37 s. MVSTATE-REUSE1 gives about 11.02 s excluding persistence and 4.27 s for REPAIR. The one-time state-cache write is about 0.18 s, yielding a fresh-chain time near 11.19 s and a cumulative speedup of about 2.44x relative to the 27.26 s PERFBASE1-era chain. Peak RSS increases about 5.6% because exact rung state is retained, while remaining far below campaign limits.

The post-gate reprofile finds no further material duplicated reconstructible CPU state: MVSEL and REPAIR are now dominated by the exact sequential sparse-state update arithmetic itself. The CPU optimization program is therefore closed. Further accelerator/runtime qualification belongs to `FINAL-GPU1`.

Progress and observability contract

Every stage expected to run long enough to appear stalled SHALL expose three layers:

Scientific progress

- completed/total domains, profiles, families, blocks, configurations, witnesses, or edges as appropriate;
- global percentage and ETA.

Executor state

- busy/allocated workers;
- ready, in-flight, and buffered tasks;
- memory-budget use where measurable.

Current hot items

- identities of slow/active families, shards, or proposals;
- local progress for a long single item.

A heartbeat is emitted even when no task completes during the reporting interval. ETA is based on global committed work, not one current profile.

MLFF progress presentation grammar

As of `0.20.237a0`, every user-facing MLFF progress/heartbeat message SHALL use the same presentation grammar. This is presentation state only and SHALL NOT enter scientific digests or execution-cache identity.

- Dynamic progress fields appear in the order `status`, `progress`, `elapsed`, `eta`, rate fields, then stage-specific telemetry.
- `elapsed` and known `eta` SHALL be fixed-width `HH:MM:SS`; durations longer than 99 hours retain all hour digits. Unknown/not-yet-estimable ETA SHALL be exactly `--:--:--`. Humanized alternatives such as `39m44s`, `27.9 min`, `10s`, or `estimating` are forbidden in MLFF progress output.
- Counted work SHALL use `progress=completed/total (percent%)`, with thousands separators for large counters. A stage without a meaningful total SHALL report `status=phase; phase=...` rather than inventing a percentage.
- Throughput SHALL carry an explicit stable unit such as `frame/s`, `witness/s`, `task/s`, or `edge/s`. When both are available, the recent/current rate precedes the cumulative average rate.
- Fields SHALL be semicolon-delimited. Stage prefixes such as `[DATA6 sweep]`, `[TRAIN run-id]`, or `[EVALUATION scheduler]` identify the emitter but do not replace the canonical fields.
- Scheduler heartbeats SHALL report the same elapsed/ETA grammar and expose completed progress plus active/pending/queue telemetry rather than using a separate prose-only dialect.
- Cache restoration, phase transitions, and rung events use the same `status=...; progress=...` or `status=phase; phase=...` vocabulary where applicable.

The shared helpers in `mdstats.training_data.progress_timing` own duration, fraction, rate, and timing-field formatting so individual stages do not reintroduce private ETA dialects.

Performance qualification

Every performance gate is measured at worker counts 1, 2, a bounded intermediate count, and automatic full budget. The qualification record includes:

- wall time and CPU time;
- effective occupancy U_P;
- throughput in domain-appropriate units;
- peak RSS and persisted bytes;
- queue occupancy/backpressure telemetry;
- output/content digest;
- exact scientific-record equality.

For MVSEL, equivalence is checked after every selected rank on bounded fixtures. For REPAIR, the entire accepted/rejected swap trace is compared. For MVIDX, every CSR/CSC offset and index array is compared between reuse and full-rebuild paths.

PERFBASE1 frozen baseline authority

`PERFBASE1` is implemented as a measurement-only, versioned record. Scientific-output identity is separated from execution telemetry so later exact-equivalent implementations may change wall time, CPU occupancy, memory layout, worker count, and queue behavior without changing the scientific

baseline digest. The record binds the foundation family/variant/checkpoint SHA-256 as an input identity but does not encode MPA-0-specific behavior; MH-1 and other supported foundations use the same record contract.

The revision-92 CPU evidence uses the supplied LTA target archive and unified 12,000-frame replay source plus deterministic synthetic FEAS1/MVIDX1/MVSEL1 workloads. The supplied TARGET-DATA2B radius workload is a fixed 4,100-frame, eight-family representative cache spanning low/high temperature and hydrostatic strain; the complete 27-file target archive is authenticated separately. On the qualification host the automatic CPU budget is three lanes, so the bounded-intermediate schedule aliases the two-lane schedule. Stages that are serial in the current implementation record the requested schedule separately from actual allocated lanes rather than reporting fictitious parallelism.

The canonical evidence is `benchmarks/mlff_perfbase1_lta_cloud_cpu_mpa0_2026-08-17.{json,md}`. All repeated trials preserve exact scientific-output digests. The active MPA-0 medium checkpoint is bound by SHA-256 `75428afe3a1d...fb493e38604fb638`. MACE model inference is not claimed on the cloud host because that runtime was not part of the authoritative measurement environment; Foundation Audit/EVAL2 inference baselines remain explicitly unavailable there rather than being synthesized.

Multi-billion-edge MVIDX out-of-core hardening

As of `0.20.238a0`, campaign MVIDX MUST NOT require the complete candidate-to-witness inverse edge payload plus full-family SciPy transpose workspace to coexist in anonymous RAM. When the inverse payload for a family exceeds the execution threshold, MVIDX performs deterministic source-row chunk transposes, appends each candidate column in ascending source-row chunk order, and writes the exact `<u4` candidate-witness array directly to an NPY memmap. Candidate offsets remain canonical `<u8`. Chunk size and concurrent family count are execution-only and are admitted under the explicit `StageResourceScope` RAM budget. The out-of-core result SHALL be byte-identical to the in-memory deterministic transpose.

Whole-array NPY memmaps may be hard-linked into the native MVIDX record when source and destination share a filesystem; this is persistence reuse, not scientific identity. The campaign SHALL reload the durable native record before transient build paths are removed. Required inverse disk capacity is preflighted from exact edge cardinality, and MVIDX reports canonical `HH:MM:SS` elapsed/ETA heartbeats during long inversion.

Part VII - Current implementation status and frozen forward gates

Current authority snapshot

The current campaign architecture retains the scientific contracts established by DATA1-DATA9, conventional nested CV, target/replay evaluation, MACE adapter/version locks, deployment verification, TARGET-DATA2 multi-view selection, and FINAL-GPU1 deferral. Revision 99 completes `MVQUAL-PAR1`: independent same-N domain/selector/size qualification jobs are globally scheduled under PARCORE1 while every TARGET-DATA2B report, MVIDX cross-check, hard-obligation decision, comparison order, and persisted MVQUAL record remains unchanged.

The most recent implemented performance gates are:

- **COVREF-PAR1 (`0.20.229a0`)** - TARGET-DATA2B exact reference-radius construction on one single-level global block queue with adaptive cache-sized tasks, O(1) pair/species adapters, and unchanged radius/reference authority.
- **MVKERNEL1 (`0.20.230a0`)** - exact ragged-CSR/vector telemetry kernels around the unchanged sequential MVSEL rank authority.
- **REPAIR-PAR1 (`0.20.231a0`)** - vectorized immutable repair proposals with adaptive deterministic proposal parallelism and unchanged sequential repair winner authority.
- **MVQUAL-PAR1 (`0.20.232a0`)** - global deterministic same-N scoring queue with one native numerical lane/job and canonical post-queue comparison reduction.

`PERFBASE1` (`0.20.225a0`) remains the frozen measurement authority used to judge subsequent optimization gates. `TARGET-DATA2B-FEAS1-PERF3` (`0.20.223a0`) is the direct predecessor scheduler whose successful global single-level execution pattern PARCORE1 generalizes.

The multi-view scientific gates FEAS1, MVIDX1, MVSEL1, REPAIR1, MVPERF1, MVQUAL1, SIZE-HALVE2, SIZE-FIDELITY2, and MVMIGRATE1 remain the governing target-data design. The new optimization gates below are exact-equivalence execution work layered on top of those authorities.

Frozen campaign optimization sequence

Gate PERFBASE1 - reproducible performance baselines - COMPLETE

Purpose. Freeze representative supplied-data and synthetic workloads before broad optimization.

Implementation. `0.20.225a0` adds the foundation-generic `PerfBase1Record`/workload/trial schemas, stage meters, deterministic benchmark harness, exact output-drift rejection, and Markdown/JSON evidence rendering. Requested and actually allocated workers are recorded separately so current serial stages are not misreported as parallel.

Record. The canonical MPA-0 CPU authority is stored as the PERFBASE1 JSON record and Markdown report under `benchmarks/`. It authenticates the supplied 27-file target archive, fixed representative target-family cache, unified 12,000-frame replay source, dependencies, active foundation checkpoint, and benchmark implementation manifest. It records wall/CPU time, assigned-lane occupancy, RSS, throughput, worker settings, queue telemetry where available, and exact scientific-output digests. The schema is not MPA-0-specific and can bind an MH-1 checkpoint unchanged.

Observed baseline. On the cgroup-limited cloud CPU (automatic budget three lanes), FEAS1 median wall time changes from about 1.78 s at one worker to 0.85 s at three workers, while current MVIDX1 changes from about 2.17 s to 2.40 s and occupancy falls from about 1.02 to 0.33. TARGET-DATA2B reference radii improve from about 0.61 s to 0.42 s. MVSEL rank authority and replay ExtXYZ ingest correctly remain single-lane baselines.

Acceptance. PASS. Every repeated schedule preserves the workload scientific-output digest exactly; inputs are SHA/content-addressed; repeated wall-time CV is low enough for implementation comparison on all but the sub-second three-lane radius probe, which remains usable as a coarse scaling indicator. Foundation Audit/EVAL2 model-inference baselines are explicitly unavailable on this cloud environment rather than fabricated.

Succeeded by. `PARCORE1` in `0.20.226a0`.

Gate PARCORE1 - shared deterministic CPU scheduler - COMPLETE

Purpose. Replace repeated bespoke executors with one bounded, resource-aware, deterministic work substrate.

Implementation. `0.20.226a0` adds `DeterministicWorkQueue`, `DeterministicWorkItem`, `DeterministicWorkCompletion`, queue snapshots, task-identity errors, and `DeterministicOrderedReducer`. `StageResourceScope` now carries the stage RAM budget as well as CPU/native-thread limits. Ready, submitted/in-flight, and completed work are independently bounded; persistent memory can be reserved/released explicitly; queue and memory backpressure are counted; heartbeat snapshots expose executor state. Task locality metadata is retained for later NUMA work without activating NUMA affinity. FEAS1 is migrated from its private `ThreadPoolExecutor` coordinator to the shared queue while preserving its canonical per-profile witness reduction.

Resource ownership. Campaign FEAS1 passes the explicit stage scope to the queue, which applies BLAS/OpenMP quarantine once at queue scope; cKDTree remains one native worker/task while outer work can fill the budget. Direct API callers that omit a scope keep the pre-PARCORE resource-control behavior. The queue may submit up to twice the executing-lane count to hide coordinator hand-off latency, but simultaneous executing lanes remain exactly bounded by `python_workers`.

Acceptance. PASS. All queue contract tests pass, FEAS1 scientific output retains the exact PERFBASE1 digest (SHA-256 prefix `937214c70d1f2baa`, full value in the canonical qualification record) across worker schedules, and all three automatic-budget lanes are observed active without nested oversubscription. In the final paired same-host two-repeat comparison, PARCORE1 records a three-worker median of about 0.83 s versus about 0.94 s for the untouched `0.20.225a0` implementation; assigned-lane occupancy is about 0.66 versus 0.64. The PARCORE1 full-budget result is also consistent with the frozen PERFBASE1 median of about 0.85 s. Serial, dual, and bounded-intermediate paired medians are likewise non-regressive in the final pair. Timing is treated as execution evidence rather than scientific authority; the exact digest is the gate invariant.

Succeeded by. `NEIGHBOR1` in `0.20.227a0`.

Gate NEIGHBOR1 - shared FEAS1/MVIDX exact-neighborhood engine - COMPLETE

Purpose. Compute exact feature-family neighborhoods once and reuse them.

Implementation. `0.20.227a0` adds the shared exact-neighborhood engine and content-addressed forward-neighborhood store. FEAS1 emits streamed canonical witness CSR while preserving its historical support/capacity reduction order; ragged cKDTree results are compressed at the worker boundary and discarded after canonical commit. Worker count, query-block size, and queue settings are execution-only and excluded from cache identity. Native-array persistence authenticates manifests/arrays and campaign restart validates the cache independently. The exact final CSR allocation is RAM-admitted before materialization. MVIDX1 consumes authenticated forward CSR directly; its source no longer contains cKDTree/query-ball geometry. Cache miss/staleness rebuilds once through the same global exact engine. The existing CSR-to-CSC inversion remains unchanged.

Acceptance. PASS. On the PERFBASE1 synthetic authority (6 families, 49,152 witnesses, 3,194,880 exact edges), FEAS1, the NEIGHBOR1 cache, and cached/rebuilt MVIDX1 remain digest-identical across worker/block settings. Their SHA-256 values begin `937214c70d1f`, `0220c89084fe`, and `e408bd25dcc9`; the complete values are frozen in the gate qualification record. A cache-hit regression test replaces

the geometric query method with a fail-fast sentinel and still passes, proving zero second geometric queries. Native-array round trip/tamper tests pass. On the cgroup-limited three-lane cloud CPU, final two-repeat medians reduce FEAS1->MVIDX1 wall time from about 2.77 s in untouched `0.20.226a0` to about 1.03 s with NEIGHBOR1 (about 2.68x end-to-end); one-worker total improves from about 2.20 s to about 1.28 s. Timing is execution evidence only; digest equality is the authority.

Succeeded by. `MVIDX-REUSE1` in `0.20.228a0`.

Gate MVIDX-REUSE1 - stable parallel sparse inversion - COMPLETE

Purpose. Reduce MVIDX to authenticated graph adoption, deterministic CSR-to-CSC inversion, and obligation metadata.

Implementation. `0.20.228a0` schedules independent required-family inverse builds and hard-obligation inversion through `DeterministicWorkQueue` under one `StageResourceScope`. Each individual transpose uses the deterministic compiled SciPy counting transpose with one native lane; canonical family order is restored after arbitrary task completion, so no atomics or nested sparse-kernel threads are required. The measured row-by-row sorted/unique validator is replaced by one vectorized adjacent-index comparison with CSR-boundary masking. The originally sketched Python-threaded intra-family degree/prefix/range fill was implemented experimentally and rejected because it was slower than the compiled kernel on the frozen workload.

Acceptance. PASS. The frozen MVIDX digest (SHA-256 prefix `e408bd25dcc9`; full value in the qualification record) is unchanged across one-, two-, and three-lane schedules, with byte-identical candidate offsets and candidate-to-witness arrays. On the same cgroup-limited cloud CPU, untouched `0.20.227a0` cached MVIDX measured about 0.59 s median, while revision 95 measured about 0.16 s at one lane, 0.12 s at two lanes, and 0.087 s at three lanes; the final paired three-lane gain is about 6.8x. Timing is execution evidence only; sparse-array/digest equality is scientific authority.

Next gate. `COVREF-PAR1`.

Gate COVREF-PAR1 - TARGET-DATA2B exact CPU parallelization - COMPLETE

Purpose. Remove the remaining one-driver cKDTree pattern from reference-radius/coverage construction.

Implementation. `0.20.229a0` routes exact local-radius row blocks through the stage-wide `DeterministicWorkQueue`, with one native cKDTree worker/task and one shared read-only tree/scaled matrix per family. Execution-only adaptive row sizing caps the historical block size by an approximately 2 MiB query-temporary target and a minimum task-count rule, preventing long tails on high-core-count hosts. Pair-rule and foundation-species adapters use O(1) maps, and target-label scalar channels apply their exact historical constant-family rejection before expensive statistics/tree work. Direct API calls without an execution scope retain the historical native-tree implementation for qualification/backward compatibility.

Acceptance. PASS. The frozen PERFBASE1 supplied-data radius digest remains exactly `823a2c0c2f8a...6d96e52cd642e2d` across 1/2/3 outer lanes. In the final same-host four-repeat supplied-cache comparison, untouched `0.20.228a0` measures about 0.279 s median at three native cKDTree workers and `0.20.229a0` measures about 0.223 s at three outer lanes (about 1.25x faster); all three outer lanes are observed active. The frozen PERFBASE1 three-lane baseline was about 0.42 s, illustrating host-load variability and why paired controls are retained. A separate 36,408-row nonuni-

form equal-unit/equal-frame two-repeat stress family preserves byte-identical radii and improves from about 5.94 s to about 5.13 s (about 1.16x). One-lane queue execution is essentially neutral/slightly slower on the small cache, so the gate does not claim a serial speedup. The exact scientific/reference arrays, not wall time, are the gate authority; nested native-tree parallelism is rejected.

Next gate. `MVKERNEL1`.

Gate MVKERNEL1 - sparse selector/qualification vector kernels - COMPLETE
Purpose. Reduce Python overhead in MVSEL/MVQUAL without altering sequential rank decisions.

Implementation. `0.20.230a0` adds shared exact ragged-CSR gather kernels and routes MVSEL inverse-edge updates through them. Per-family and domain-total gain arrays share one gathered edge stream, while `np.add.at` is still applied independently in canonical witness/edge order so floating-point state remains exact. Coverage and representative witness amounts are vectorized, and required hard-obligation pending count is maintained incrementally. MVIDX selected-subset coverage and obligation helpers gather candidate CSR rows once and use boolean assignment/`bincount`. MVQUAL gathers selected candidate rows once per family to derive multiplicity, covered/unique witness masks, and unique-owner candidates; DATA2A run/condition provenance codes are built once per domain. The scalar MVSEL update and scalar MVQUAL telemetry implementations are retained as qualification references.

Acceptance. PASS. Optimized and scalar MVSEL states agree exactly after every qualified rank and persisted selection plans remain byte-identical. The frozen 4,096-candidate/2,048-selection digest remains `d147d85acd64...b2ffaddbb978b378`; the 24,576-candidate/16,384-selection stress digest remains `aaec42fb0c1d...9a0461bcd75d608`. Same-host measurements reduce the representative selector median from about 1.404 s in untouched `0.20.229a0` to about 0.811 s, and the 16,384-selection stress path from about 6.640 s to about 5.591 s. A 16,384-candidate/8,192-selected/6-family MVQUAL telemetry fixture drops from about 0.578 s to about 0.041 s (about 14.1x), with byte-identical telemetry. Full MVQUAL plan evidence remains digest-identical to untouched `0.20.229a0`. Timing is execution evidence; exact selector state and qualification records are authority.

Production-density maintenance (`0.20.241a0`). COMPLETE. A 36,408-candidate, 165-family, 9.51-billion-edge campaign exposed unbounded selector temporaries hidden by the smaller qualification fixtures. Exact complete-row initialization chunks now cap indexed FP64 weight gathers at 512 MiB; scanned inverse mmap pages are released; numerical guards use a candidate-sized touched bitmap; and families at least 98% dense use canonical witness-ordered contiguous-run adds. Scoped initialization plus rank 0 improves from about 320 s to 106 s (about 3.03x), with peak RSS reduced from about 51.5 GiB to 19.2 GiB. Focused scalar-oracle and bit-exact kernel tests pass. Threaded scatter remains rejected after only about 1.1x measured gain, and no parallel rank authority is introduced.

Next gate. `REPAIR-PAR1`.

Gate REPAIR-PAR1 - deterministic parallel repair proposals - COMPLETE
Purpose. Reduce REPAIR1 proposal cost without changing the sequential repair iteration, swap objective, tie hierarchy, accepted/rejected trace, or winner application.

Implementation. `0.20.231a0` fuses removal unique-coverage and representative-loss scans, scores complete replacement frontiers with shared ragged-CSR gathers, and replaces repeated set intersections with thread-private epoch/stamp witness membership. A candidate-to-rank inverse map makes

future displacement lookup O(1). Immutable removal proposals may run through the PARCORE1 deterministic queue; each task owns private stamp scratch, native numerical layers remain single-threaded, and results are canonically reduced in the historical removal-shortlist order. Parallel dispatch is adaptive: proposal batches below an execution-only sparse-edge threshold remain serial because qualification showed that blindly threading the historical Python loops is slower. The winning pair's representative contribution is recomputed with the historical scalar/stamp arithmetic before the swap is persisted. `execution_mode="reference"` retains the historical scalar proposal oracle.

Acceptance. PASS. The complete serialized repair plan/trace on the frozen REPAIR1 fixture is identical for the scalar reference and optimized 1/2/4-worker realizations, digest `5dcb048b02ae...b265a52615b9545b`. A 2,048-candidate proposal fixture preserves result digest `1a09e7745aa5...244421e4b859a9b1` and improves from about 3.176 s in untouched `0.20.230a0` to about 0.119 s (about 26.6x); adaptive execution correctly keeps it serial. An 8,192-candidate sparse fixture preserves result digest `9fda146806fc...10c468b41994` and improves from about 3.130 s to about 0.830/0.611/0.461 s at 1/2/4 lanes: about 3.77x from vectorization alone, 6.79x end-to-end at four lanes, and 1.80x additional 1-to-4-lane scaling. Timing is execution evidence; the complete repair trace and terminal order are scientific authority.

Next gate. `MVQUAL-PAR1`.

Gate MVQUAL-PAR1 - global same-N scoring queue - COMPLETE

Purpose. Execute independent domain/selector/size rescoring concurrently without changing any same-N qualification authority.

Implementation. `0.20.232a0` freezes one execution-only job for every materializable `(domain, selector, target size)` score. A job performs the existing immutable TARGET-DATA2B rescore, MVKERNEL1 batched sparse telemetry, hard-obligation state, and MVIDX covered-mass cross-check. Large job sets execute through the PARCORE1 `DeterministicWorkQueue`; completion order is arbitrary, but comparisons and progress messages are reconstructed afterward in the historical domain/size order. Campaign jobs force cKDTree and BLAS/OpenMP to one native lane each, preventing nested oversubscription. Per-job temporary-memory estimates participate in queue admission. Automatic campaign mode is capped at four outer score lanes because qualification shows these jobs become memory-bandwidth limited; an explicit larger override remains available for qualified high-bandwidth hosts. Direct API calls without an explicit `StageResourceScope` deliberately do not alter the process native-thread environment. This preserves historical direct-API behavior while campaign execution retains its pre-gate BLAS=1 scientific authority; qualification caught that changing only BLAS thread count can shift the Wasserstein diagnostic by about 1e-16 and therefore change a cryptographic report digest.

Acceptance. PASS. Scalar/direct and 1/2/4-worker realizations preserve complete qualification-plan dictionaries under their respective historical native-thread contract. Under the production BLAS=1 contract, untouched `0.20.231a0` and MVQUAL-PAR1 preserve the same plan digest `2ebd7f5dc2b5...befda74059fc90b` on a 16,000-reference, six-size, 12-job same-N fixture. Same-host warm medians are about 0.866 s for the old serial driver with four native cKDTree workers and about 0.409 s for four outer MVQUAL lanes with one native tree worker/job (about 2.12x faster). The new path measures about 0.828/0.451/0.458 s at 1/2/4 outer lanes; all four lanes are observed active and all

12 jobs complete without queue or memory backpressure on the fixture. Timing is execution evidence; exact qualification records and digests remain scientific authority.

Next gate. `AUDIT-EVAL-PERF1`.

Gate AUDIT-EVAL-PERF1 - Foundation Audit and EVAL2 CPU hardening - COMPLETE

Purpose. Remove repeated Python frame/species/statistics loops surrounding already optimized model inference without changing any prediction, checkpoint-selection, or GPU numerical authority.

Implementation. `0.20.233a0` adds an execution-only EVAL2 static-reduction cache keyed by the immutable evaluation-view object and ordered correlation-block IDs. It precomputes composition keys, per-frame species membership, focus masks, and compact block codes once, then reuses them across checkpoint reductions. Force-tail vector storage is preallocated rather than accumulated as ragged per-frame arrays and concatenated. Paired block bootstrap preserves the exact seeded NumPy RNG stream but draws in bounded vector batches sized from a 32 MiB temporary target, eliminating the 2,000-replicate Python loop without unbounded allocation. FOUNDATION-AUDIT1 now builds the DATA3 frame-array index once for the whole audit, shares immutable per-run species membership across audit domains, reuses `delta*delta` work for total/species reductions, and evaluates all configured force-tail quantiles in one call. Prediction-manifest authentication and conditioned-feature semantics are unchanged; no new model inference is introduced.

Acceptance. PASS. On a 4,096-frame / 294,912-atom repeated EVAL2 fixture, untouched `0.20.232a0` and `0.20.233a0` preserve exact metric digest `d9dd9db2c2d4...9d3f15762d434658`; same-host median reduction time improves from about 0.862 s to 0.449 s (about 1.92x). On a 768-block, 2,000-replicate paired bootstrap, the exact comparison digest `9664354fd2d8...14acb729590397e` is preserved while median time improves from about 0.0512 s to 0.0152 s (about 3.36x). The available no-inference FOUNDATION-AUDIT1 fixture preserves audit digest `39b8b207c741...61ba87b0e8c94e5`, keeps model-provider descriptor/prediction call counts fixed at 44/44, and improves median audit reduction from about 0.0580 s to 0.0545 s (about 1.06x). Timing is execution evidence; persisted metric/audit/bootstrap records remain scientific authority.

Next gate. `REPLAY-PERF1`.

Gate REPLAY-PERF1 - replay index/cache and chunk materialization - COMPLETE

Purpose. Avoid repeated serial parsing of the immutable replay corpus without changing REPLAY-UNIFY1 source, label, split, prediction, or retention authority.

Implementation. `0.20.234a0` adds `ReplaySourceIndex`, a reconstructible execution artifact bound to the exact source SHA-256, source-artifact digest, source-order geometry digest, byte size, frame offsets/lengths, and atom counts. The cache is stored beneath the campaign replay-internal tree, is independently authenticated on reuse, relocates without scientific change when identical source bytes move, and is rebuilt automatically on source mutation or receipt corruption. Indexed readers seek only requested source indices; contiguous requested frames are parsed in bounded chunks, while sparse monitor reconstruction does not scan unrelated frames. True-label materialization, pseudo-label materialization, and foundation-prediction cache construction reuse the authenticated source-order geometry identity instead of recomputing canonical geometry hashes after parsing. ExtXYZ parsing itself remains single-threaded: measured Python-threaded ASE parsing was slower, so REPLAY-PERF1 does not introduce a misleading parser thread pool.

Acceptance. PASS. On the supplied 12,000-frame replay corpus (`187eed42...98403c`), the persisted index has content digest `ce6c678a...c0c5e1`; first byte-index construction is about 0.45 s and an authenticated restart hit about 0.07 s on the qualification host. Monitor-only true-label reconstruction preserves logical digest `633aae8a...bf1114` and exact ExtXYZ SHA-256 `cc0f9b30...eab2cf` while improving median wall time from about 9.14 s to 3.01 s (~3.03x). A complete parse/geometry-identity pass preserves the ordered-identity digest while improving from about 7.64 s to 6.42 s (~1.19x). Full train+monitor materialization preserves byte-identical train and monitor files and improves more modestly from about 15.68 s to 14.35 s (~1.09x), as expected because every frame must still be parsed and written. Cache corruption and source mutation are fail-safe reconstruction events, and chunk size does not enter scientific identity.

Next gate. `CAMPAIGN-PERF-QUAL1`.

Gate CAMPAIGN-PERF-QUAL1 - end-to-end optimization closure - COMPLETE

Purpose. Reprofile the accumulated CPU optimization program as an integrated campaign rather than summing isolated microbenchmarks, while preserving all scientific and GPU numerical authority.

Implementation. `0.20.235a0` is a measurement/documentation release: runtime scientific algorithms are unchanged from `0.20.234a0`. The closure runs a common 8,192-candidate/six-family FEAS1 -> NEIGHBOR1 -> MVIDX1 -> MVSEL1 -> REPAIR1 -> MVQUAL1 chain against an untouched `0.20.225a0` PERFBASE1-era control, rechecks the supplied 12,000-frame replay restart path, and reproduces EVAL2/bootstrap and Foundation Audit CPU records. Exact output digests, restart/cache identities, worker scaling, process CPU, and peak RSS are recorded in `benchmarks/mlff_campaign_perf_qual1_cloud_cpu_mpa0_2026-08-17.json`.

Acceptance. PASS, FOLLOW-UP REQUIRED. The integrated target-data chain improves from about 27.26 s to a four-lane median of about 11.95 s (~2.28x) with exact FEAS1/MVIDX1/MVSEL1/REPAIR1/MVQUAL1 scientific digests. The current one/two/four-lane chain is about 12.91/12.07/11.95 s, showing that the remaining tail is no longer parallel-starved. Peak RSS rises from about 306 MiB to about 343 MiB because authenticated sparse execution state is retained, but remains far below the campaign memory ceiling with no observed backpressure. Replay monitor reconstruction reproduces exact bytes with an authenticated index hit near 0.07 s; EVAL2 and paired-bootstrap closure reruns reproduce their frozen digests. Reprofiling shows MVSEL exact sparse state mutation consumes most selector time, while REPAIR performs approximately 4,098 additional `_select_and_update` calls to reconstruct the already-computed selected-order state and spends several seconds on that replay before/around proposal scoring. This is duplicated reconstructible execution work, so total CPU optimization closure is deferred one targeted gate.

Next gate. `MVSTATE-REUSE1`.

Gate MVSTATE-REUSE1 - selector-to-repair sparse-state reuse - COMPLETE

Purpose. Remove duplicated selector-state reconstruction inside REPAIR while preserving every selector rank, target rung, repair objective/tie decision, swap, terminal order, and MVQUAL record.

Implementation. `0.20.236a0` adds authenticated exact MVSEL state checkpoints at each materializable rung and a native bundled-array store. REPAIR restores a checkpoint only before its first accepted repair swap; after repair diverges, it carries the historical mutable state forward. Pure checkpoint reconciliation after divergence was measured and rejected because it changed FP64 representative-gain

entries at roughly `1e-17–1e-16`. Bounded post-divergence CSR gather batching changes preparation only; all state mutations remain candidate-major in the historical arithmetic order. Fresh campaign execution passes the just-built cache directly from MVSEL to REPAIR while persisting it for restart. Invalid cache state fails safely to exact replay.

Acceptance. PASS; CPU OPTIMIZATION CLOSED. On the common 8,192-candidate/six-family fixture, the untouched 0.20.235a0 chain median is about 12.00 s. MVSTATE-REUSE1 is about 11.02 s excluding persistence and about 11.19 s including the one-time cache write (~1.07x fresh-chain speedup); REPAIR improves about 5.37 s -> 4.27 s (~1.26x). All FEAS1/NEIGHBOR1/MVIDX1/MVSEL1/REPAIR1/MVQUAL1 digests remain exact, cache restart/tamper/stale-lineage behavior is qualified, and peak RSS remains within budget. Relative to the PERFBASE1-era 27.26 s chain, the fresh accumulated CPU realization is about 2.44x faster. The residual tail is exact sequential sparse-state arithmetic rather than another material duplicate execution artifact.

`0.20.238a0` hardens the already-complete MVIDX-REUSE1 execution path for production caches containing billions of exact NEIGHBOR1 edges. Large inversions are now file-backed and chunked under explicit RAM admission, with exact byte-equivalence to the in-memory transpose. This is an execution/storage maintenance adaptation under revision 103 and does not reopen the CPU optimization gate sequence.

Next gate. `FINAL-GPU1` workstation qualification.

Explicit non-goals

The optimization program does not authorize approximate neighborhood search, approximate coverage, learned subset selectors, GPU graph authority, relaxed 0.95 coverage, larger-than-16,384 rescue, altered CV leakage boundaries, or new locked-test tuning. GPU qualification remains consolidated at the final release boundary.

Documentation and lineage rule

Current scientific/execution contracts belong in this manual or module specifications. Revision comments and release deltas belong only in `docs/history/mlff/`. A historical note may explain why a decision changed, but it may not override the current manual. Every future architecture revision SHALL update the history index and this manual's current-state section rather than prepending another revision block to the document.

Part VIII - Ownership boundaries and decision summary

Physical-observable validation ownership boundary

Physical observable calculation is not owned by `mdstats.training_data`. RDF, coordination, neighbor-angle statistics, connectivity, topology statistics, MSD, VACF, spectra, VDOS, diffusion, displacement distributions, current correlations, and ionic conductivity remain authoritative in their respective `mdstats.analysis` modules and architecture manuals.

The MLFF branch owns only:

1. choosing an advisory observable-recommendation profile and an explicit recipe;
2. constructing an immutable recipe of analysis call IDs and parameters;
3. running the same recipe on matched reference and MLFF collections;

4. preserving verified collection and frame-selection identity, symmetric reference and candidate trajectory-generation identity, runtime/capability identity, warning records, and analysis-owned result identities;
5. binding every execution to an explicit statistical role and, where required, to a predeclared comparison policy, protocol freeze, and test-activation record;
6. applying comparison and acceptance policies only after those policies are frozen and independently identified.

It does not own the numerical algorithms, normalization, neighbor definitions, plateau estimators, spectral transforms, or graph statistics.

The analysis-owned standardized facade is `mdstats.analysis.observable_validation`. The MLFF-owned bridge delegates to that facade and stores no duplicate scientific arrays or algorithms.

The initial ObservableRecommendationProfile values are:

- `generic_condensed`, `crystalline_solid`, and `amorphous_solid`;
- `liquid` and `interface`.

These are advisory call sets, not automatic material classifiers. The user still supplies species/groups, cutoffs, projections, trajectory windows, thermodynamic conditions, and any interface coordinate. Ionic transport is an explicit extension. Porous, zeolite, ring, cage, and site calls are optional extensions and must never be activated merely because the reference application is LTA.

Selection features versus validation observables

Compact structural descriptors used for partitioning or frame selection are MLFF workflow inputs. Full physical observables used to judge a trained model remain analysis products. An MLFF feature provider may call a lower-level analysis primitive when that primitive has an explicit per-frame contract, but it must record the owner API and cannot redefine the observable. Expensive trajectory observables such as diffusion, VDOS, conductivity, or residence statistics are validation jobs, not ordinary frame-selection features.

Implemented call boundary in 0.20.44a0 and consistency closure in 0.20.45a0

The first standardized recipe registry covers the implemented general structural and dynamical calls, including RDF, coordination, bond angles, atomic connectivity/statistics, MSD, VACF, velocity spectra, VDOS, VACF diffusion, diffusion plateau selection, van Hove, non-Gaussian dynamics, self-intermediate scattering, charge current, current correlation, ionic conductivity, and Nernst-Einstein comparison. Native result dataclasses remain owned by the analysis modules.

The 0.20.45a0 closure validates recipe dependencies at construction, preflights machine-readable collection requirements, records versioned capability/codec identity, captures warnings, per-call durations, and runtime versions, and binds candidate model and MD protocol identity to paired evidence. DATA9A6c in 0.20.46a0 strengthens this contract: supplied collection identities are recomputed and verified; location hints do not alter scientific identity; reference and candidate generation records must both bind the output collection; each native result receives an analysis-owned canonical digest; statistical role and locked-test activation are explicit; and comparison-policy identity is upstream of realized evidence. Comparison metrics and scientific acceptance thresholds remain a future MLFF policy layer; call execution alone is not a pass/fail judgment. Static EOS, elasticity, finite-temperature

response, viscosity, phonons, surfaces, interfaces, defects, and migration barriers are owned by `thermomechanical_energetic_validation_architecture.md`.

Statistical role, policy ordering, and locked-test leakage

Physical-observable evidence is assigned one explicit role: `training_diagnostic`, `checkpoint_monitor`, `outer_validation`, `calibration`, `locked_test`, or `external_benchmark`. The role is not inferred from a filename or caller context.

A comparison policy is a predeclared object. The allowed dependency order is:

```
ObservableComparisonPolicy
    +
ObservableValidationActivationRecord
    +
Reference/Candidate Collection and Generation Identities
    -> ObservableValidationEvidence
    -> ObservableComparisonResult
    -> ObservableAcceptanceDecision
```

The reverse edge is forbidden. Realized RDFs, diffusion coefficients, phonons, or other physical results must not be inspected to choose their own acceptance thresholds. A locked-test activation record additionally requires the frozen training protocol, partition assignment, and explicit evaluation activation. Locked-test observable evidence cannot alter feature fitting, selection, training protocol, checkpoint selection, calibration policy, or acquisition. The dependency graph represents this role specialization explicitly as `LOCKED_TEST_OBSERVABLE_EVIDENCE`; ordinary checkpoint-monitor evidence is not globally forbidden from later policy-governed checkpoint assessment.

`ObservableValidationEvidence` stores analysis-owned result identities, not a second scientific result schema. The authoritative analysis module remains responsible for serializing or identifying its native result. The MLFF layer references that identity when comparing reference and candidate outputs.

Required module specifications

Before each runtime stage, write or revise specifications for:

```
sampling/autocorrelation
sampling/blocks
sampling/assignment
training_data/sources
training_data/label_domains
training_data/reference_energies
training_data/feature_metric
training_data/identity
training_data/eligibility
training_data/conditions
training_data/strain
training_data/events
training_data/features/base
training_data/material_profiles
training_data/atom_groups
training_data/profile_features
training_data/profile_events
```

```
training_data/features/lta  # optional compatibility profile
training_data/observable_comparisons
training_data/features/mace
training_data/partition
training_data/cross_validation
training_data/checkpoint_selection
training_data/independence
training_data/selection
training_data/exposure
training_data/replay
training_data/replay_retention
training_data/active_learning
training_data/role_inheritance
training_data/export/extxyz
training_data/export/mace
training_data/workflow
```

Decision summary

The branch follows ten scientific rules.

1. **Independent evidence remains independent.** Cross-validation uses fresh models, nested checkpoint monitors, and evaluation folds that never control checkpoint choice.
2. **The complete training protocol is the comparison unit.** Replay, objective, checkpoint, and exposure choices are part of cross-validation identity.
3. **Selection and E0 fitting are training-domain local.** Transforms, fitted metrics, selection, residual difficulty, and atomic-reference corrections do not inspect held-out evidence.
4. **Physical facts and workflow decisions are separate.** Occurrence, geometry, labels, policies, fitted products, and runtime realizations remain distinct.
5. **Data and deformation conventions are explicit.** Label domains, stress, energy channels, E0 limitations, and ASE cell-matrix conventions are audited.
6. **Declared focus physics receives explicit coverage.** Profile events, atom-group environment quotas, group-resolved metrics, and rare transitions cannot be hidden by abundant host statistics. LTA/mobile-ion semantics are an optional specialization.
7. **Weights and exposure are audited.** Selection, property loss, head balance, and actual MACE loader duplication are separate records.
8. **Locked tests are operationally sealed.** Activation requires frozen protocol and committee identities.
9. **Replay and uncertainty policies are enforced.** Candidate checkpoints obey target/group/replay constraints, and calibration is bound to the actual final committee and an applicability domain.
10. **Expansion is append-only by default.** Active-learning children inherit existing roles and add new cohorts without silently rewriting old evidence.

References

[1] I. Batatia, D. P. Kovacs, G. N. C. Simm, C. Ortner, and G. Csanyi, "MACE: Higher Order Equivariant Message Passing Neural Networks for Fast and Accurate Force Fields," *Advances in Neural Information Processing Systems* **35**, 11423-11436 (2022). DOI: 10.48550/arXiv.2206.07697.

[2] ACEsuit, "MACE descriptors," MACE documentation. Available at: https://mace-docs.readthedocs.io/en/latest/guide/descriptors.html (accessed 2026-07-27).

[3] H. Flyvbjerg and H. G. Petersen, "Error Estimates on Averages of Correlated Data," *Journal of Chemical Physics* **91**, 461-466 (1989). DOI: 10.1063/1.457480.

[4] J. Racine, "Consistent Cross-Validatory Model-Selection for Dependent Data: hv-Block Cross-Validation," *Journal of Econometrics* **99**, 39-61 (2000). DOI: 10.1016/S0304-4076(00)00030-0.

[5] D. R. Roberts, V. Bahn, S. Ciuti, et al., "Cross-Validation Strategies for Data with Temporal, Spatial, Hierarchical, or Phylogenetic Structure," *Ecography* **40**, 913-929 (2017). DOI: 10.1111/ecog.02881.

[6] J. D. Morrow, J. L. A. Gardner, and V. L. Deringer, "How to Validate Machine-Learned Interatomic Potentials," *Journal of Chemical Physics* **158**, 121501 (2023). DOI: 10.1063/5.0139611.

[7] VASP Software GmbH, "Smearing technique," VASP Wiki. Available at: https://vasp.at/wiki/Smearing_technique (accessed 2026-07-27).

[8] ACEsuit, "Training," MACE documentation. Available at: https://mace-docs.readthedocs.io/en/latest/guide/training.html (accessed 2026-07-27).

[9] ACEsuit, `mace-torch` 0.3.16, Python Package Index, released 2026-05-10. Available at: https://pypi.org/project/mace-torch/0.3.16/ (accessed 2026-07-27).

[10] ACEsuit, `estimate_e0s_from_foundation`, MACE reference implementation, version-locked by the adapter at implementation time. Current source available at: https://github.com/ACEsuit/mace/blob/main/mace/data/utils.py (accessed 2026-07-27).

[11] ACEsuit, "Multihead Replay Finetuning," MACE documentation. Available at: https://mace-docs.readthedocs.io/en/latest/guide/multihead_finetuning.html (accessed 2026-07-27).

[12] ACEsuit, "Multihead Training for MACE," MACE documentation. Available at: https://mace-docs.readthedocs.io/en/latest/guide/multihead_training.html (accessed 2026-07-27).

[13] C. Schran, K. Brezina, and O. Marsalek, "Committee Neural Network Potentials Control Generalization Errors and Enable Active Learning," *Journal of Chemical Physics* **153**, 104105 (2020). DOI: 10.1063/5.0016004.

[14] A. R. Tan, S. Urata, S. Goldman, J. C. B. Dietschreit, and R. Gomez-Bombarelli, "Single-Model Uncertainty Quantification in Neural Network Potentials Does Not Consistently Outperform Model Ensembles," *npj Computational Materials* **9**, 225 (2023). DOI: 10.1038/s41524-023-01180-8.

[15] I. Batatia, P. Benner, Y. Chiang, et al., "A Foundation Model for Atomistic Materials Chemistry," *Journal of Chemical Physics* **163**, 184110 (2025). DOI: 10.1063/5.0297006.

[16] M. Kulichenko, B. Nebgen, N. Lubbers, J. S. Smith, et al., "Data Generation for Machine Learning Interatomic Potentials and Beyond," *Chemical Reviews* **124**, 13681-13714 (2024). DOI: 10.1021/acs.chemrev.4c00572.

[17] ACEsuit, `mace.tools.train`, MACE version 0.3.16 source, especially the validation-head iteration and last-head checkpoint rule. Available at: https://github.com/ACEsuit/mace/blob/v0.3.16/mace/tools/train.py (accessed 2026-07-27).

[18] ACEsuit, `mace.cli.run_train`, MACE version 0.3.16 source, especially multi-head assembly, replay-ratio duplication, head ordering, and loader construction. Available at: https://github.com/ACEsuit/mace/blob/v0.3.16/mace/cli/run_train.py (accessed 2026-07-27).

[19] ACEsuit, `mace-torch` version 0.3.16 `setup.cfg`, complete runtime `install_requires` contract. Available at: https://github.com/ACEsuit/mace/blob/v0.3.16/setup.cfg (accessed 2026-07-28).

[20] e3nn developers, `e3nn` 0.4.4 package metadata and dependency contract, Python Package Index. Available at: https://pypi.org/project/e3nn/0.4.4/ (accessed 2026-07-28).

[21] R. Kern, "A Simple File Format for NumPy Arrays," NumPy Enhancement Proposal 1, 2007. Available at: https://numpy.org/doc/1.13/neps/npy-format.html (accessed 2026-08-15).

[22] NumPy developers, "numpy.load," NumPy reference documentation. Available at: https://numpy.org/doc/stable/reference/generated/numpy.load.html (accessed 2026-08-15).

[23] National Institute of Standards and Technology, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, 2015. DOI: 10.6028/NIST.FIPS.180-4.

[24] R. J. Hyndman and Y. Fan, "Sample Quantiles in Statistical Packages," *The American Statistician* **50**(4), 361–365 (1996). DOI: 10.1080/00031305.1996.10473566.

[25] J. L. Bentley, "Multidimensional Binary Search Trees Used for Associative Searching," *Communications of the ACM* **18**(9), 509–517 (1975). DOI: 10.1145/361002.361007.

[26] SciPy developers, "scipy.spatial.cKDTree," SciPy reference documentation. Available at: https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.html (accessed 2026-08-15).

[27] T. F. Gonzalez, "Clustering to Minimize the Maximum Intercluster Distance," *Theoretical Computer Science* **38**, 293–306 (1985). DOI: 10.1016/0304-3975(85)90224-5.

[28] SciPy developers, "scipy.spatial.cKDTree.query," SciPy reference documentation. Available at: https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.query.html (accessed 2026-08-15).

[29] SciPy developers, "scipy.stats.wasserstein_distance," SciPy reference documentation. Available at: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.wasserstein_distance.html (accessed 2026-08-15).

[30] K. Jamieson and A. Talwalkar, "Non-stochastic Best Arm Identification and Hyperparameter Optimization," *Proceedings of AISTATS*, PMLR 51:240–248, 2016. Available at: https://proceedings.mlr.press/v51/jamieson16.html.

[31] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization," *Journal of Machine Learning Research* **18**(185), 1–52 (2018). Available at: https://www.jmlr.org/papers/v18/16-558.html.

[32] R. D. Blumofe and C. E. Leiserson, "Scheduling Multithreaded Computations by Work Stealing," *Journal of the ACM* **46**(5), 720-748 (1999). DOI: 10.1145/324133.324234.

[33] SciPy developers, `scipy.spatial.cKDTree.query_ball_point`, SciPy reference documentation. Available at: https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.query_ball_point.html (accessed 2026-08-17).

[34] SciPy developers, compressed sparse row/column matrix documentation, including `scipy.sparse.csr_matrix` and `scipy.sparse.csc_matrix`. Available at: https://docs.scipy.org/doc/scipy/reference/sparse.html (accessed 2026-08-17).

[35] `threadpoolctl` developers, "Python helpers to limit native thread pools," project documentation and source. Available at: https://github.com/joblib/threadpoolctl (accessed 2026-08-17).

[36] J. Deters, J. Wu, Y. Xu, and I.-T. A. Lee, "A NUMA-Aware Provably-Efficient Task-Parallel Platform Based on the Work-First Principle," arXiv:1806.11128 (2018). Available at: https://arxiv.org/abs/1806.11128.

[37] NumPy developers, `numpy.bincount`, NumPy reference documentation. Available at: https://numpy.org/doc/stable/reference/generated/numpy.bincount.html (accessed 2026-08-17).